

电商系统 MVP 设计方案与技术理解

姓名: Lawrence Zhou 日期: 2026年6月

一、用户角色与核心场景 (User Roles & Core Scenarios)

1.1 项目背景

产品目标

随着线上购物模式的普及，电商平台已经成为消费者购买商品的重要渠道。为了满足消费者在线浏览、购买商品以及商家进行商品运营和订单管理的需求，公司计划从零搭建一套基础电商系统 (E-commerce System)。

该系统需要同时满足两类用户的核心需求：

消费者侧需求

消费者希望能够通过平台快速完成商品选购和购买流程，包括：

1. 浏览商品列表
2. 查看商品详情
3. 搜索和筛选商品
4. 加入购物车
5. 提交订单
6. 在线支付
7. 查询订单状态
8. 跟踪物流信息

消费者关注的核心问题是：

- 商品信息是否清晰完整
- 下单流程是否简单流畅
- 支付过程是否稳定可靠
- 订单状态是否透明可追踪

因此，系统需要提供完整且稳定的购买链路，帮助用户顺利完成交易。

商家运营侧需求

除消费者外，平台还需要支持运营人员完成日常业务管理工作，包括：

1. 商品创建与维护
2. SKU管理
3. 库存管理
4. 订单管理
5. 发货管理
6. 售后处理

运营人员关注的核心问题是：

- 商品是否能够快速上架销售
- 库存是否准确可控
- 订单是否能够高效处理
- 发货流程是否顺畅

因此，系统需要建设基础运营后台，支撑商品销售及订单履约过程。

MVP范围

本项目为第一阶段产品建设(MVP, Minimum Viable Product)，目标是在最短时间内验证电商交易闭环是否能够正常运行，而非一次性建设完整成熟的电商平台。

因此，本阶段仅保留支撑核心交易流程所必需的功能。

核心交易流程如下：





通过以上流程，系统能够完成从商品展示到交易完成的完整业务闭环。

MVP建设原则

为了保证项目能够快速上线并验证业务价值，本次MVP设计遵循以下原则：

1. 优先保证核心交易链路可用

任何影响用户完成购买流程的功能均优先开发，包括：

- 商品展示
- SKU选择
- 购物车
- 订单创建
- 支付能力
- 发货管理

确保用户能够成功完成一次完整购买。

2. 优先保证数据准确性

电商业务涉及库存、价格、订单金额等关键数据。

系统必须保证：

- 库存准确
- 订单金额准确
- 商品状态准确
- 支付状态准确

避免出现超卖、少卖或订单状态异常等问题。

3. 优先保证后台运营能力

商品无法管理、库存无法维护，即使前台功能完善也无法正常销售。

因此后台商品管理、库存管理及订单管理能力属于MVP必备功能。

4. 控制开发复杂度

MVP阶段不追求功能丰富度, 而追求快速验证商业模式。

因此以下能力暂不纳入首期建设范围:

- 会员体系
- 优惠券系统
- 营销活动系统
- 商品评价系统
- 个性化推荐系统
- 直播带货能力
- 社交分享能力

上述功能将在后续版本根据业务发展情况逐步规划。

产品成功标准

MVP上线后, 产品应能够满足以下目标:

用户侧目标

1. 用户能够独立完成商品浏览和购买流程
2. 用户能够查询订单状态
3. 用户能够查看物流信息

运营侧目标

1. 运营人员能够独立管理商品
2. 运营人员能够维护库存
3. 运营人员能够处理订单及发货

业务侧目标

1. 完成完整交易闭环验证
2. 支撑平台初期商品销售
3. 收集用户行为数据和业务数据
4. 为后续产品迭代提供依据

通过以上目标，本项目将完成基础电商平台从商品展示到订单履约的核心能力建设，为后续业务增长和功能扩展奠定基础。

1.2 用户角色定义

为了保证系统设计能够覆盖核心业务场景，需要明确系统的主要用户角色及其使用目标。根据电商平台业务特点，本系统主要涉及三类用户：

- 1. 消费者 (Customer / Buyer)
- 2. 运营人员 (Operations Staff / Merchant Staff)
- 3. 平台管理员 (Platform Admin)

不同角色关注的目标不同，因此需要提供不同的功能支持。

1.2.1 消费者 (Customer / Buyer)

用户描述

消费者是电商平台的核心用户，也是平台收入的直接来源。
消费者通过平台浏览商品、比较商品信息、完成下单支付，并在购买后跟踪订单状态和物流信息。
对于消费者而言，平台的核心价值在于能够方便、快捷、安全地完成购物过程。

用户目标

消费者希望：

- 1. 快速找到需要购买的商品
- 2. 获取准确完整的商品信息
- 3. 以合理价格完成购买
- 4. 及时了解订单状态
- 5. 顺利收到商品
- 6. 在出现问题时获得退款或售后支持

用户痛点

痛点	说明
----	----

商品信息不完整	无法判断商品是否符合需求
搜索效率低	难以快速找到目标商品
库存信息不准确	下单时发现缺货
支付流程复杂	导致用户放弃购买
订单状态不透明	用户无法了解物流进度
售后流程复杂	降低用户满意度

核心使用场景

场景一：浏览商品

用户进入平台后浏览商品列表，了解当前在售商品。

涉及功能：商品列表页，分类筛选，商品搜索

场景二：查看商品详情

用户进入商品详情页查看商品信息。

涉及功能：商品图片展示，商品名称，商品价格，SKU规格，库存信息，销量展示

场景三：加入购物车

用户将多个商品加入购物车，统一进行结算。

涉及功能：加入购物车，修改购买数量，删除商品

场景四：立即购买

用户直接购买单个商品。

涉及功能：SKU选择，地址确认，提交订单

场景五：支付订单

用户通过在线支付完成购买。

涉及功能：支付页面，支付状态反馈

场景六: 订单跟踪

用户查询订单处理进度。

涉及功能: 订单列表, 订单详情, 物流查询

场景七: 退款申请

用户在符合条件时申请退款。

涉及功能: 发起退款, 查看退款进度

用户价值总结

对于消费者而言, 系统需要提供从商品发现到订单完成的一站式购物体验, 并保证:

- 商品信息透明
- 库存信息准确
- 交易流程顺畅
- 订单状态可追踪

1.2.2 运营人员 (Operations Staff)

用户描述

运营人员负责商品销售过程中的日常运营工作, 是保证平台正常运行的重要角色。

运营人员通过后台系统管理商品、库存、订单及发货流程, 确保消费者能够顺利完成购物。

用户目标

运营人员希望:

1. 快速创建商品
2. 灵活维护商品信息
3. 实时管理库存
4. 高效处理订单
5. 及时安排发货
6. 快速处理异常订单

用户痛点

痛点

说明

商品维护效率低

商品更新耗时

库存管理混乱

导致超卖或缺货

订单量增加后处理困难

影响履约效率

发货流程复杂

增加运营成本

缺乏数据支持

难以分析业务情况

核心使用场景

场景一：创建商品

运营人员创建新商品并录入商品信息。

涉及功能：商品创建，图片上传，商品分类设置，SKU设置

场景二：编辑商品

运营人员更新商品价格、库存或状态。

涉及功能：商品编辑，商品上下架

场景三：库存管理

运营人员维护商品库存。

涉及功能：库存调整，库存查询，库存预警

场景四：订单管理

运营人员查看和处理订单。

涉及功能：订单查询，订单状态管理，异常订单处理

场景五：安排发货

运营人员完成发货操作。

涉及功能：创建物流单，填写物流单号，更新发货状态

场景六：处理退款

运营人员审核退款申请。

涉及功能: 退款审批, 退款状态更新

用户价值总结

对于运营人员而言, 系统需要提供高效、稳定的运营后台, 帮助其完成商品销售和订单履约工作, 提高运营效率并降低人工成本。

1.2.3 平台管理员 (Platform Admin)

用户描述

平台管理员负责整个系统的管理和维护, 主要关注平台运行情况、权限管理以及异常问题处理。在MVP阶段, 平台管理员并非直接参与交易流程, 但需要保证系统稳定运行。

用户目标

平台管理员希望:

- 1. 管理后台账号权限
- 2. 查看平台整体运营情况
- 3. 监控关键业务指标
- 4. 处理异常订单和系统问题

用户痛点

痛点	说明
权限管理混乱	存在安全风险
缺乏整体数据视图	难以掌握运营情况
异常订单排查困难	影响用户体验
系统问题发现滞后	增加运营风险

核心使用场景

场景一: 用户与权限管理
管理后台账号及角色权限。

涉及功能:账号管理, 角色管理, 权限配置

场景二:数据监控

查看平台经营数据。

涉及功能:数据看板, 核心指标统计

场景三:异常订单处理

处理交易过程中的异常情况。

涉及功能:异常订单查询, 状态修复, 人工干预

用户价值总结

对于平台管理员而言, 系统需要提供完善的管理能力和数据监控能力, 保证平台安全、稳定、高效运行。

1.3 用户角色关系总结

用户角色	核心目标	主要功能模块
消费者	完成商品购买	商品、购物车、订单、支付
运营人员	管理商品与订单	商品管理、库存管理、订单管理、发货管理
平台管理员	管理平台运行	用户管理、权限管理、数据监控

三类角色共同构成完整的电商业务闭环：
消费者产生订单需求；
运营人员完成商品销售与订单履约；
平台管理员保障平台稳定运行。
通过明确各角色职责与需求, 可以为后续MVP功能设计和优先级划分提供依据。

二、MVP功能清单与优先级设计

2.1 优先级定义

在MVP阶段，产品设计的核心目标不是一次性实现完整电商平台的所有能力，而是在有限时间和资源下，优先保证核心交易链路能够正常运行。

本系统的核心交易流程为：

浏览商品 → 加入购物车 / 立即购买 → 提交订单 → 支付 → 发货 → 完成订单

因此，功能优先级将围绕该交易闭环进行划分。

优先级	定义	判断标准
P0	必须上线功能	如果缺少该功能，用户无法完成核心交易流程，或运营人员无法支撑 商品销售和订单履约
P1	建议上线功能	不影响基础交易闭环，但可以明显提升用户体验、运营效率或异常处理能力
P2	后续迭代功能	对交易闭环不是必需，更多用于提升增长、营销、留存或精细化运营能力

P0功能判断逻辑

P0功能主要服务于“能不能完成一笔订单”。

例如：

- 没有商品列表，用户无法发现商品；
- 没有商品详情页，用户无法判断是否购买；
- 没有提交订单功能，交易无法生成；
- 没有支付功能，平台无法完成收款；
- 没有后台商品管理，平台没有商品可卖；
- 没有库存管理，系统容易出现超卖或缺货问题；
- 没有订单管理和发货管理，订单无法履约。

因此，P0功能必须优先建设。

P1功能判断逻辑

P1功能主要服务于“能不能更顺畅、更高效地完成交易”。

例如：

- 商品搜索和分类筛选可以提高用户查找商品的效率；
- 取消订单和退款功能可以处理异常交易场景；
- 数据看板可以帮助运营人员了解业务情况；

- 用户管理可以支持基础的平台治理。

这些功能虽然不是核心交易闭环的必要条件，但会影响用户体验、运营效率和平台稳定性，因此适合放在P1。

P2功能判断逻辑

P2功能主要服务于“能不能提升增长和长期运营能力”。

例如：

- 优惠券可以促进转化，但不是基础交易必须功能；
- 商品评价可以提高用户信任，但MVP阶段可以暂缓；
- 会员体系、推荐系统、营销活动等功能可以提升复购和用户粘性，但开发复杂度较高，不适合第一阶段投入过多资源。

因此，P2功能适合在核心交易链路稳定后再进行规划。

2.2 用户端功能清单与优先级

用户端功能主要面向消费者，目标是帮助用户完成从商品浏览到支付下单的完整购物流程。

模块	功能	优先级	原因
账号系统	登录 / 注册	P0	用户需要通过账号识别身份，系统需要关联购物车、订单、支付记录和售后记录。没有账号体系，用户订单难以管理。
商品中心	商品列表	P0	商品列表是用户发现商品的主要入口。没有商品列表，用户无法浏览平台商品。
商品中心	商品详情页	P0	用户需要查看商品名称、价格、图片、SKU、库存等信息后才能做出购买决策。
商品中心	商品状态展示	P0	用户需要知道商品是否上架、是否售罄。下架或无库存商品应限制购买，避免无效订单。
SKU选择	SKU选择	P0	多规格商品需要用户选择具体SKU，例如颜色、尺寸等。订单必须绑定具体SKU才能扣减库存和履约。

购物车	加入购物车	P0	购物车支持用户暂存商品, 并支持多个商品统一结算, 是基础交易流程中的重要功能。
购物车	修改购买数量	P0	用户需要在下单前调整购买数量, 系统也需要根据购买数量校验库存。
购物车	删除商品	P1	用户可以删除不想购买的商品, 提升购物车管理体验, 但不影响基础购买链路。
订单系统	提交订单	P0	提交订单是从购物行为转化为交易行为的关键步骤, 系统需要在此阶段生成订单。
订单系统	订单金额计算	P0	系统需要根据商品价格、购买数量、运费等信息计算订单金额, 保证支付金额准确。
地址系统	收货地址填写 / 选择	P0	实物商品必须有收货地址才能完成发货, 因此地址信息是订单履约的必要条件。
支付系统	在线支付	P0	支付是完成交易闭环的关键节点。没有支付功能, 订单无法转化为有效成交。
支付系统	支付结果展示	P0	用户需要明确知道支付是否成功, 系统也需要根据支付结果更新订单状态。
订单系统	订单列表	P0	用户需要查看自己的历史订单和当前订单状态。
订单系统	订单详情	P0	用户需要查看订单商品、金额、地址、支付状态、物流状态等完整信息。
物流系统	物流信息查看	P1	用户可以查看发货状态和物流进度, 提升订单透明度。MVP可先展示基础物流单号和发货状态。
搜索系统	商品搜索	P1	搜索可以帮助用户快速找到目标商品, 提高购物效率, 但在商品数量较少的MVP阶段不是绝对必要。
搜索系统	分类筛选	P1	分类和筛选可以提升商品发现效率, 但可以在商品列表基础功能完成后再优化。

售后系统	取消订单	P1	用户未支付或未发货前可能需要取消订单，有助于处理异常交易场景。
售后系统	退款申请	P1	退款是售后场景的重要能力，但在MVP阶段可以先支持简单退款流程，不需要复杂售后规则。
营销系统	优惠券	P2	优惠券有助于提升转化率，但不影响核心交易链路，可放在后续营销版本中建设。
用户互动	商品评价	P2	商品评价可以提升用户信任，但MVP阶段可先聚焦交易闭环，后续再增加评价能力。
推荐系统	个性化推荐	P2	推荐系统对数据和算法依赖较高，开发成本较高，不适合第一版MVP。

2.3 后台运营功能清单与优先级

后台运营功能主要面向运营人员和商家员工，目标是支撑商品上架、库存维护、订单处理和发货履约。

模块	功能	优先级	原因
商品管理	商品创建	P0	商品必须先被创建后才能在前台展示和销售，是电商系统的基础功能。
商品管理	商品编辑	P0	运营人员需要修改商品名称、价格、图片、描述等信息，保证商品信息准确。
商品管理	商品上下架	P0	商品状态会影响用户是否可以购买。下架商品不能继续销售，避免产生无效订单。
商品管理	商品图片管理	P0	商品图片直接影响用户购买决策，是商品详情页的重要展示内容。

SKU管理	SKU创建	P0	对于多规格商品, 系统需要通过SKU区分不同颜色、尺寸或型号, 并分别管理价格和库存。
SKU管理	SKU价格维护	P0	不同SKU可能存在不同价格, 系统需要支持SKU级别的价格维护。
库存管理	库存维护	P0	库存是下单校验和订单履约的关键数据。如果库存不准确, 可能导致超卖或用户下单失败。
库存管理	库存扣减	P0	用户下单或支付后, 系统需要根据业务规则扣减库存, 保证库存数据准确。
库存管理	库存预警	P1	当库存低于一定阈值时提醒运营补货, 可以提升运营效率, 但不是MVP核心交易闭环的必要功能。
订单管理	订单列表	P0	运营人员需要查看所有订单, 并根据订单状态进行处理。
订单管理	订单详情	P0	运营人员需要查看订单商品、用户信息、收货地址、支付状态等信息, 才能完成履约。
订单管理	订单状态管理	P0	系统需要支持订单从待支付、已支付、已发货到已完成的状态流转。
发货管理	创建发货信息	P0	已支付订单需要进入发货流程, 运营人员必须能够填写物流信息并发货。
发货管理	物流单号录入	P0	物流单号用于用户查询物流状态, 也是订单履约的重要信息。
发货管理	发货状态更新	P0	发货后订单状态需要从已支付更新为已发货, 保证用户和运营看到一致状态。
售后管理	退款审核	P1	退款处理属于售后服务能力, 对用户体验重要, 但可在MVP阶段做简化处理。
用户管理	用户账号查询	P1	后台可以查询用户信息, 便于客服或运营处理订单问题, 但不是基础交易闭环必需。

权限管理	后台角色权限	P1	不同后台人员应拥有不同操作权限，例如商品管理、订单管理、管理员权限等，有助于降低误操作风险。
数据看板	订单数据统计	P1	数据看板帮助运营人员了解订单量、支付转化和GMV等指标，但第一阶段可先做基础统计。
数据看板	商品销售数据	P1	帮助运营判断哪些商品销量较好，为后续商品运营提供依据。
营销管理	优惠券管理	P2	优惠券属于营销能力，不影响基础交易闭环，可后续迭代。
内容管理	首页Banner管理	P2	首页运营位可以提升转化，但第一阶段不是必须功能。
精细化运营	用户分层管理	P2	用户分层适合平台有一定用户规模后建设，MVP阶段暂不优先。

2.4 MVP核心功能范围总结

根据以上优先级分析，第一版MVP应优先覆盖以下核心功能：

用户端核心功能

功能	是否纳入MVP	说明
登录 / 注册	是	支持用户身份识别和订单关联
商品列表	是	支持用户浏览商品
商品详情页	是	支持用户查看商品信息并做出购买决策
SKU选择	是	支持多规格商品购买
加入购物车	是	支持用户暂存商品并统一结算
提交订单	是	支持用户生成订单

在线支付	是	支持交易完成
订单列表 / 订单详情	是	支持用户查看订单状态
物流信息查看	可简化纳入	MVP阶段可先展示发货状态和物流单号
取消订单 / 退款	可简化纳入	MVP阶段优先支持未支付订单取消和基础退款申请
优惠券 / 商品评价 / 推荐	否	属于后续增长和体验优化功能

后台核心功能

功能	是否纳入MVP	说明
商品创建与编辑	是	支持商品上架销售
商品上下架	是	控制商品是否可购买
SKU管理	是	支持多规格商品
库存管理	是	支持下单校验和库存扣减
订单管理	是	支持订单处理
发货管理	是	支持订单履约
用户管理	可简化纳入	支持基础账号查询
数据看板	可简化纳入	MVP阶段可先展示核心订单和销售数据
营销管理	否	后续版本建设

2.5 本期不纳入MVP的功能说明

为了控制开发范围并保证核心交易链路尽快上线，以下功能暂不纳入第一版MVP。

功能	暂不纳入原因	后续规划
优惠券系统	不影响基础购买流程，且涉及优惠规则、订单金额计算和营销配置，复杂度较高	可在交易链路稳定后作为营销模块建设
会员体系	不影响用户完成基础购买，且需要积分、等级、权益等额外规则设计	可在用户规模增长后建设
商品评价	有助于建立信任，但MVP阶段可以先通过商品详情信息支持购买决策	可在订单完成后增加评价入口
个性化推荐	需要用户行为数据和推荐算法支持，初期数据不足	可在平台积累浏览和购买数据后建设
营销活动	涉及活动规则、活动库存、活动价格和活动页面，开发成本较高	可作为后续增长版本规划
直播带货	不属于基础电商交易能力，且对内容运营和技术能力要求较高	暂不考虑
社交分享	可提升传播，但不影响用户完成购买	后续根据增长需求增加

2.6 功能优先级设计总结

第一版MVP的设计重点是保证电商平台能够完成最基础、最重要的交易闭环。

从用户侧来看，系统需要支持消费者完成：

浏览商品 → 查看详情 → 选择SKU → 加入购物车 / 立即购买 → 提交订单 → 支付 → 查看订单

从运营侧来看，系统需要支持运营人员完成：

创建商品 → 维护SKU和库存 → 管理订单 → 安排发货 → 更新订单状态

因此，P0功能主要围绕商品、库存、订单、支付和发货展开；P1功能主要用于提升用户体验和运营效率；P2功能则更多服务于后续增长和精细化运营。

这样的优先级划分可以保证产品在第一阶段以较低复杂度快速上线，同时为后续功能扩展保留空间。

三、核心流程设计

本部分选择两个核心流程进行详细设计：

1. 用户下单与支付流程
2. 商品创建与发布流程

选择这两个流程的原因是：用户下单与支付流程是电商系统最核心的交易链路，直接影响订单转化和收入；商品创建与发布流程则是交易链路的前置基础，如果商品信息、SKU、库存和状态管理不完整，前台交易流程也无法正常运行。

3.1 用户下单与支付流程

3.1.1 流程目标

用户下单与支付流程的目标是支持消费者从选择商品到完成支付，并生成有效订单。

该流程需要保证：

- 商品处于可购买状态
- SKU选择有效
- 库存充足
- 订单金额计算准确

- 支付状态能够正确同步
- 订单状态能够按照业务规则流转

该流程是电商系统的核心交易链路，直接决定用户是否能够完成购买。

3.1.2 涉及角色

角色	说明
消费者	选择商品、提交订单并完成支付
系统	校验商品状态、库存、价格、地址和支付结果
支付平台	处理用户支付请求并返回支付结果
运营人员	后续处理已支付订单并安排发货

3.1.3 前置条件

用户进入下单流程前，需要满足以下前置条件：

1. 用户已登录系统。
2. 商品处于上架状态，即商品状态为 ON_SALE。
3. 用户已选择有效SKU。
4. 用户购买数量大于0。
5. SKU库存数量大于或等于用户购买数量。
6. 用户已填写或选择有效收货地址。
7. 商品价格和库存信息能够被系统正常读取。

如果上述条件不满足，系统应阻止用户提交订单，并向用户展示明确的错误提示。

3.1.4 正常流程

场景：用户从商品详情页点击“立即购买”并完成支付

步骤	用户操作 / 系统动作	说明
1	用户进入商品详情页	查看商品图片、名称、价格、SKU、库存等信息
2	用户选择SKU	例如选择颜色、尺寸、型号等
3	用户选择购买数量	系统根据SKU库存判断是否可购买

4	用户点击“立即购买”	进入订单确认页
5	系统校验商品状态	判断商品是否仍为上架状态
6	系统校验SKU状态和库存	判断SKU是否有效, 库存是否充足
7	用户确认收货地址	地址用于后续发货
8	系统计算订单金额	包括商品金额、运费、优惠金额等。本MVP阶段可先不接入优惠券
9	用户点击“提交订单”	系统创建订单
10	系统生成订单号	订单状态变为“待支付”
11	用户进入支付页面	选择支付方式并发起支付
12	支付平台处理支付请求	返回支付成功或失败结果
13	支付成功	系统更新订单状态为“已支付”
14	用户查看支付结果	页面展示“支付成功”
15	订单进入待发货流程	运营人员后续安排发货

3.1.5 状态变化

订单状态变化

业务节点	订单状态	说明
用户提交订单成功	PENDING_PAYMENT(待支付)	订单已生成, 但用户尚未完成支付
用户支付成功	PAID(已支付)	支付平台返回支付成功, 订单进入待发货阶段
用户未在规定时间内支付	CANCELLED(已取消)	系统自动取消订单
用户取消未支付订单	CANCELLED(已取消)	用户主动取消订单
支付后申请退款	REFUNDING(退款中)	用户发起退款申请
退款完成	REFUNDED(已退款)	退款处理完成

3.1.6 库存变化规则

库存处理是下单流程中非常关键的部分。MVP阶段建议采用“提交订单时锁定库存，支付超时后释放库存”的方式。

业务节点	库存处理方式	说明
用户浏览商品	不扣减库存	仅展示当前可售库存
用户加入购物车	不扣减库存	购物车只是暂存商品，不代表购买成功
用户提交订单	锁定库存	防止其他用户同时购买导致超卖
用户支付成功	正式扣减库存	订单成为有效成交订单
用户支付超时	释放锁定库存	避免库存被长期占用
用户取消未支付订单	释放锁定库存	订单无效，库存恢复
支付后退款	根据业务规则决定是否回补库存 若商品未发货，通常可回补库存；若已发货，则需结合退货结果处理	

说明：如果系统初期复杂度有限，也可以采用“支付成功后扣减库存”的方式，但这种方式在高并发场景下更容易出现超卖风险。因此，从产品设计角度看，提交订单时锁定库存更稳妥。

3.1.7 异常场景设计

异常场景一：库存不足

触发条件

用户提交订单时，系统发现所选SKU库存不足。

可能原因包括：

- 用户在商品详情页看到库存时还有库存，但提交订单前已被其他用户购买；
- 用户购买数量超过当前SKU库存；
- 购物车中的商品库存发生变化；
- 前端展示库存与后端实际库存不同步。

系统处理

1. 系统阻止订单创建。
2. 不进入支付流程。
3. 页面提示用户库存不足。

4. 用户可以返回商品详情页或购物车重新选择数量。

页面提示文案

当前商品库存不足，请修改购买数量或选择其他规格。

状态变化

订单未创建，因此不产生订单状态变化。

异常场景二：商品已下架，但仍存在于购物车中

触发条件

用户之前将商品加入购物车，但在用户结算前，运营人员已将商品下架。

系统处理

1. 用户进入购物车时，系统重新校验商品状态。
2. 如果商品状态为 `OFF_SALE`，则购物车中该商品标记为“已下架”。
3. 该商品不可勾选结算。
4. 用户可以删除该商品或等待商品重新上架。

页面提示文案

该商品已下架，暂时无法购买。

状态变化

订单未创建，因此不产生订单状态变化。

业务说明

购物车中的商品信息不能作为最终交易依据。系统在提交订单前必须重新校验商品状态、SKU状态、价格和库存，后端校验结果应作为最终判断依据。

异常场景三:支付失败

触发条件

用户发起支付后, 支付平台返回支付失败结果。

可能原因包括:

- 用户账户余额不足;
- 银行卡支付失败;
- 第三方支付平台异常;
- 用户主动取消支付;
- 网络异常导致支付未完成。

系统处理

1. 订单保持“待支付”状态。
2. 系统展示支付失败提示。
3. 用户可以重新发起支付。
4. 如果超过支付时限仍未支付, 系统自动取消订单。

页面提示文案

支付失败, 请重新尝试支付或更换支付方式。

状态变化

PENDING_PAYMENT(待支付) → 仍保持 PENDING_PAYMENT(待支付)

如果后续支付超时:

PENDING_PAYMENT(待支付) → CANCELLED(已取消)

异常场景四:支付超时

触发条件

用户提交订单后, 未在规定时间内完成支付。

例如: 订单创建后30分钟内未完成支付。

系统处理

1. 系统定时任务检查超时未支付订单。

2. 将订单状态更新为 **CANCELLED**。
3. 释放订单锁定库存。
4. 用户再次进入订单详情时，页面展示订单已取消。
5. 用户如仍需购买，需要重新下单。

页面提示文案

订单支付超时，已自动取消，请重新下单。

状态变化

PENDING_PAYMENT(待支付) → **CANCELLED**(已取消)

库存变化

释放该订单锁定的库存。

异常场景五：支付结果回调延迟

触发条件

用户已在支付平台完成支付，但系统暂未收到支付成功回调。

系统处理

1. 前端页面显示“支付处理中”。
2. 系统主动查询支付平台支付结果。
3. 如果确认支付成功，订单状态更新为 **PAID**。
4. 如果确认支付失败，订单保持 **PENDING_PAYMENT**。
5. 如果超过支付时限仍未确认成功，则根据支付平台最终状态决定是否取消订单。

页面提示文案

支付结果确认中，请稍后刷新查看订单状态。

状态变化

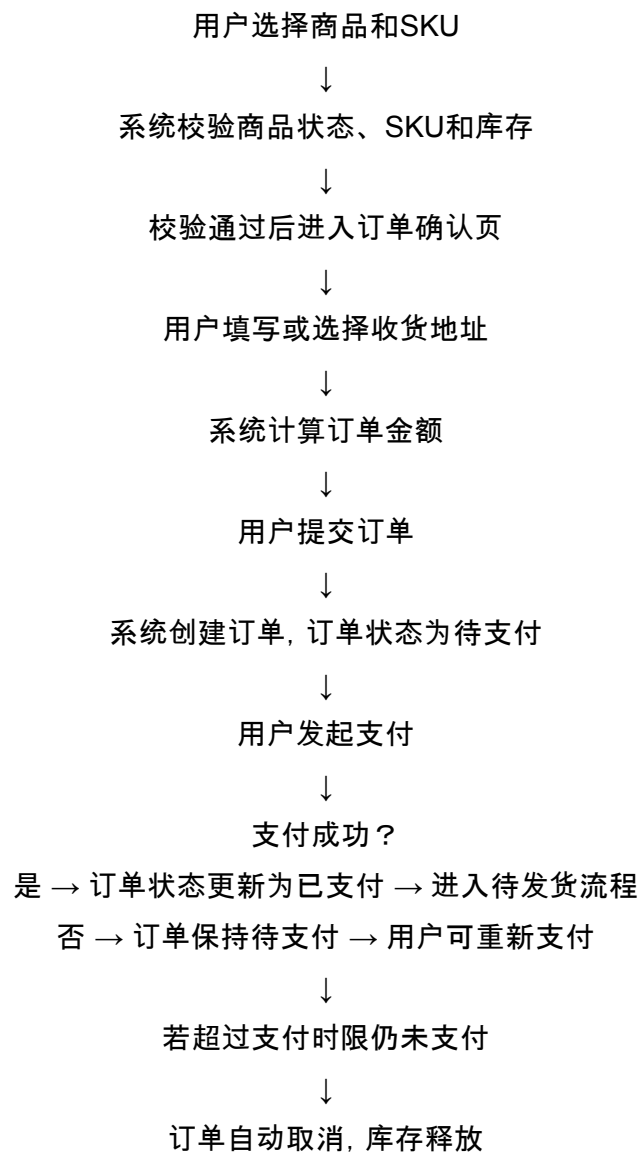
PENDING_PAYMENT(待支付) → **PAID**(已支付)

或：

PENDING_PAYMENT(待支付) → CANCELLED(已取消)

3.1.8 流程图描述

用户下单与支付流程可描述为：



3.1.9 产品设计重点

该流程设计中, 产品经理需要重点关注以下问题:

1. 后端校验必须作为最终判断依据

前端展示的商品状态和库存可能存在缓存或延迟, 因此用户提交订单时, 后端必须重新校验:

- 商品是否上架
- SKU是否有效
- 库存是否充足
- 价格是否发生变化
- 用户地址是否有效

2. 订单状态必须清晰

订单状态应能准确反映当前业务阶段, 避免出现用户已支付但订单仍显示待支付、商家已发货但用户看不到物流等问题。

3. 支付异常需要可恢复

支付失败或支付处理中不应直接让用户丢失订单, 而应允许用户重新支付或等待支付结果同步。

4. 库存处理需要防止超卖

库存不足是电商系统中最常见的问题之一。系统需要在提交订单时进行库存校验, 并根据业务规则锁定或扣减库存。

3.2 商品创建与发货流程

3.2.1 流程目标

商品创建与发布流程的目标是支持运营人员在后台创建商品, 并将商品发布到前台供消费者浏览和购买。

该流程需要保证:

- 商品基础信息完整
- 商品价格有效
- SKU信息正确
- 库存数量准确
- 商品状态可控
- 上架商品能够在前台正常展示

商品创建与发布是电商交易链路的前置流程。只有商品信息准确、库存可售、状态正常，用户端的购买流程才有基础。

3.2.2 涉及角色

角色	说明
运营人员	创建商品、维护商品信息、设置SKU和库存，并执行上架操作
系统	校验商品字段、保存商品信息、控制商品状态、同步前台展示
消费者	在前台浏览已上架商品并购买
平台管理员	在必要时管理商品权限或处理异常商品

3.2.3 前置条件

- 运营人员创建商品前，需要满足以下条件：
- 1. 运营人员已登录后台系统。
 - 2. 运营人员拥有商品管理权限。
 - 3. 商品所属类目已存在。
 - 4. 商品图片素材、商品名称、价格、库存等基础信息已准备完成。
 - 5. 若商品存在多规格，需要提前明确SKU规则，例如颜色、尺寸、型号等。

3.2.4 商品状态定义

商品在创建和发布过程中，需要通过状态控制其是否可被用户看到和购买。

商品状态	含义	前台表现
DRAFT(草稿)	商品信息尚未完善，仅后台可见	前台不可见
PENDING_REVIEW(待审核)	商品已提交，等待审核	前台不可见
ON_SALE(上架中)	商品已发布，可售卖	前台可见且可购买
OFF_SALE(已下架)	商品被运营主动下架或系统下架	前台不可购买
SOLD_OUT(已售罄)	商品库存为0	前台可展示但不可购买

说明:如果MVP阶段不设计审核机制,可以先不使用 PENDING_REVIEW 状态,运营人员创建商品后可直接保存为草稿或上架。

3.2.5 正常流程

场景:运营人员创建商品并发布上架

步骤	操作 / 系统动作	说明
1	运营人员进入后台商品管理页面	查看商品列表
2	点击“新建商品”	进入商品创建页面
3	填写商品基础信息	包括商品名称、商品类目、商品图片、商品描述等
4	设置商品价格	填写销售价和原价
5	设置SKU信息	例如颜色、尺寸、型号等
6	设置SKU价格和库存	不同SKU可以有不同价格和库存
7	系统校验商品信息	校验必填字段、价格、库存、SKU完整性
8	运营人员保存商品	商品状态为草稿
9	运营人员点击“上架”	系统再次校验商品可售条件
10	系统更新商品状态为 ON_SALE	商品进入可售状态
11	前台商品列表和详情页展示该商品	用户可以浏览和购买

3.2.6 状态变化

商品状态变化

业务节点	商品状态	说明
创建商品但未填写完整	DRAFT(草稿)	商品只在后台可见
商品信息填写完整并保存	DRAFT(草稿)	商品仍未上架

商品通过校验并点击上架	ON_SALE(上架中)	商品前台可见且可购买
运营人员主动下架	OFF_SALE(已下架)	商品前台不可购买
商品库存为0	SOLD_OUT(已售罄)	商品前台可展示, 但购买按钮不可用
运营补充库存并重新上架	ON_SALE(上架中)	商品恢复可购买

3.2.7 商品字段设计

运营人员创建商品时, 需要填写以下字段:

字段	是否必填	说明
商品名称	是	用于前台展示和搜索
商品类目	是	用于商品分类和筛选
商品图片	是	用于商品列表和详情页展示
商品描述	是	帮助用户了解商品特点
销售价	是	用户实际购买价格
原价	否	用于价格对比展示
SKU名称	是	例如白色、黑色、M码、L码等
SKU价格	是	不同SKU可能有不同价格
SKU库存	是	用于下单库存校验
商品状态	是	控制商品是否可售
运费规则	MVP可选	用于计算订单总金额
售后规则	MVP可选	用于退款和售后说明

3.2.8 业务规则

规则一: 商品名称不能为空

商品名称是用户识别商品的核心信息。如果商品名称为空, 系统不允许保存或上架。

错误提示: 商品名称不能为空。

规则二:商品至少需要上传一张图片

商品图片影响用户购买决策,也是商品列表和详情页的基础展示信息。

错误提示:请至少上传一张商品图片。

规则三:销售价必须大于0

销售价必须为有效金额,不能为0或负数。

错误提示:商品销售价必须大于0。

规则四:SKU库存不能小于0

库存数量不能为负数。若库存为0,商品或SKU不可购买。

错误提示:SKU库存不能小于0。

规则五:上架商品必须至少有一个有效SKU

如果商品没有可售SKU,则无法完成下单。

错误提示:请至少设置一个有效SKU后再上架。

规则六:商品上架前必须通过完整性校验

系统需要校验商品名称、图片、价格、SKU、库存等关键字段。如果信息不完整,商品只能保存为草稿,不能上架。

错误提示:商品信息未填写完整,暂不能上架。

规则七:下架商品不可购买

当商品状态为 OFF_SALE 时,前台应限制购买。

前台展示:该商品已下架,暂时无法购买。

规则八:库存为0时不可购买

当商品总库存或用户选择的SKU库存为0时,购买按钮置灰,不允许用户提交订单。

前台展示:当前商品库存不足。

3.2.9 异常场景设计

异常场景一：商品信息未填写完整

触发条件

运营人员点击“上架”时，系统发现商品缺少必填字段。

可能缺失字段包括：

- 商品名称
- 商品图片
- 商品价格
- SKU信息
- SKU库存
- 商品类目

系统处理

1. 系统阻止商品上架。
2. 商品状态保持为 DRAFT。
3. 页面标注缺失字段。
4. 提示运营人员补充完整信息。

提示文案

商品信息未填写完整，请完善后再上架。

状态变化

DRAFT(草稿) → 仍保持 DRAFT(草稿)

异常场景二：SKU库存为0

触发条件

运营人员设置商品SKU时，某个SKU库存为0。

系统处理

1. 允许商品保存。
2. 若所有SKU库存均为0，则不允许商品上架，或上架后展示为售罄状态。

3. 若部分SKU库存为0, 则该SKU不可购买, 其他有库存SKU仍可购买。

提示文案

该SKU库存为0, 用户暂时无法购买该规格。

状态变化

如果所有SKU库存为0:

DRAFT(草稿) → **SOLD_OUT**(已售罄)或保持 **DRAFT**(草稿)

如果部分SKU库存为0:

商品状态可为 **ON_SALE**, 但对应SKU不可购买。

异常场景三: 商品价格异常

触发条件

运营人员输入的商品价格不符合规则。

例如:

- 销售价为0
- 销售价为负数
- SKU价格为空
- 原价低于销售价且未明确允许

系统处理

1. 系统不允许上架。
2. 页面提示价格字段异常。
3. 运营人员修改价格后重新提交。

提示文案

商品价格设置异常, 请检查销售价和SKU价格。

状态变化

DRAFT(草稿) → 仍保持 **DRAFT**(草稿)

异常场景四:商品上架后被运营下架

触发条件

商品已经上架,但运营人员因为库存、价格、商品质量或业务原因主动下架商品。

系统处理

1. 商品状态更新为 **OFF_SALE**。
2. 前台商品详情页展示“商品已下架”。
3. 商品不可加入购物车,不可立即购买。
4. 已存在于用户购物车中的该商品标记为“已下架”。
5. 用户提交订单时,系统重新校验商品状态,阻止购买。

前台提示文案

该商品已下架,暂时无法购买。

状态变化

ON_SALE(上架中) → **OFF_SALE**(已下架)

异常场景五:商品上架后库存售罄

触发条件

商品销售过程中,商品总库存或某一SKU库存变为0。

系统处理

1. 商品详情页库存展示为0。
2. 对应SKU不可选择或选择后购买按钮置灰。
3. 若所有SKU库存均为0,则商品整体展示为售罄。
4. 用户无法提交订单。
5. 运营人员可在后台补充库存。

前台提示文案

当前商品已售罄。

状态变化

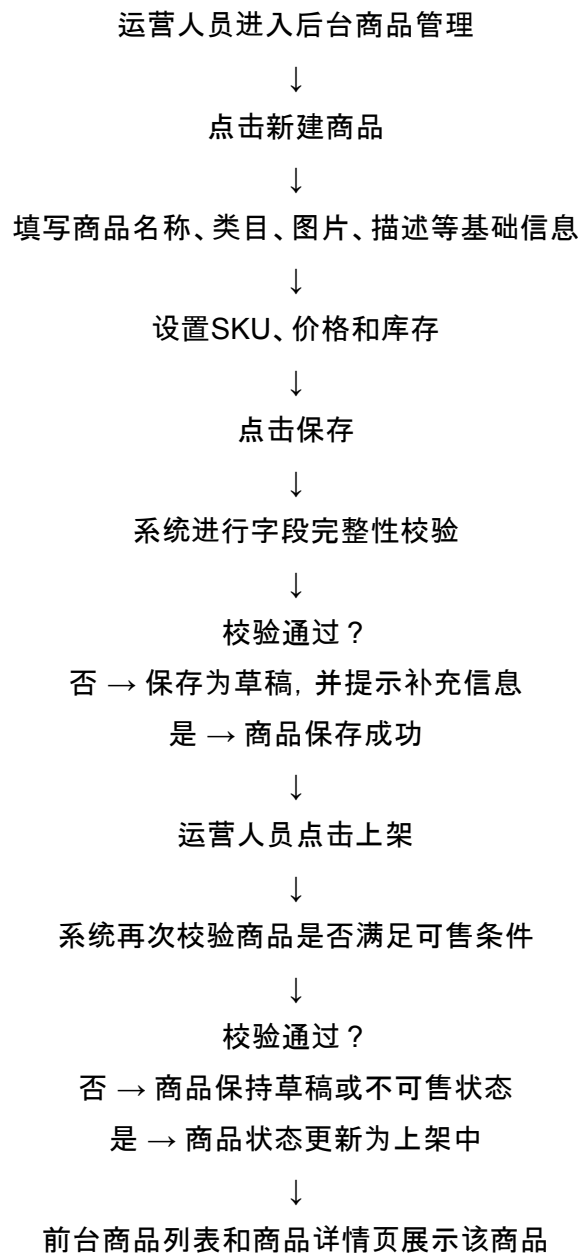
ON_SALE(上架中) → SOLD_OUT(已售罄)

补充库存后：

SOLD_OUT(已售罄) → ON_SALE(上架中)

3.2.10 流程图描述

商品创建与发布流程可描述为：



↓
用户可浏览并购买

3.2.11 产品设计重点

1. 商品状态必须明确

商品状态会直接影响前台展示和用户购买行为。

系统必须明确区分：

- 草稿商品：仅后台可见
- 上架商品：前台可见且可购买
- 下架商品：不可购买
- 售罄商品：可展示但不可购买

如果状态定义不清晰，容易导致用户看到不可购买商品，或者运营误将未完成商品发布到前台。

2. SKU是库存和价格管理的核心单位

在电商系统中，用户最终购买的通常不是一个抽象商品，而是某一个具体SKU。

例如： 商品：Bluetooth Earphones
SKU 1：White, 价格199, 库存10
SKU 2：Black, 价格209, 库存15

因此，订单、库存扣减和价格展示都应以SKU为核心单位，而不是只依赖商品总库存。

3. 上架前必须进行完整性校验

商品上架前，系统必须校验商品信息是否完整，否则可能出现以下问题：

- 商品无图片，影响用户购买判断
- 商品无价格，无法下单
- 商品无库存，无法购买
- 商品无SKU，订单无法绑定具体规格

因此，商品上架应比商品保存有更严格的校验规则。

4. 前台展示需要和后台状态保持一致

运营人员在后台修改商品状态后，前台应及时同步。例如：

- 商品下架后，前台立即禁止购买；

- SKU库存为0后, 前台应展示售罄;
- 商品价格变化后, 订单提交前必须重新校验价格。

前台展示可以允许一定缓存, 但订单提交环节必须以后端实时校验结果为准。

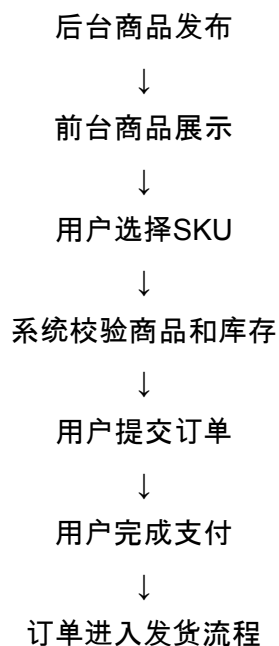
3.3 两个流程之间的关系

用户下单与支付流程依赖商品创建与发布流程。

只有当后台商品满足以下条件时, 用户端才能正常购买:

1. 商品已创建;
2. 商品状态为 `ON_SALE`;
3. 商品至少有一个有效SKU;
4. SKU库存大于0;
5. 商品价格有效;
6. 商品信息能够在前台正常展示。

两个流程之间的关系可以理解为:



因此, 商品管理流程决定“有没有商品可以卖”, 下单支付流程决定“用户能不能完成购买”。两者共同构成电商MVP的核心业务基础。

四、简化PRD设计:商品详情页

4.1 功能名称

商品详情页 (Product Detail Page)

4.2 功能背景

商品详情页是电商系统中连接“商品浏览”和“下单购买”的关键页面。

用户在商品列表中看到商品后, 需要进入商品详情页进一步了解商品信息, 包括商品图片、商品名称、价格、SKU规格、库存、销量等内容。用户只有在充分了解商品信息后, 才会决定是否加入购物车或立即购买。

因此, 商品详情页不仅承担商品展示功能, 也承担购买决策引导功能, 是影响用户转化率的重要页面。

在本MVP版本中, 商品详情页的核心目标不是实现复杂的营销展示, 而是保证用户能够清晰理解商品信息, 并顺利进入后续购买流程。

本功能需要重点解决以下问题:

1. 用户能否清楚看到商品是什么;
2. 用户能否知道商品当前价格;
3. 用户能否选择正确的SKU;
4. 用户能否知道当前商品或SKU是否有库存;
5. 用户能否在商品可售时加入购物车或立即购买;
6. 当商品下架、售罄或库存不足时, 系统能否给出明确提示。

商品详情页属于用户端P0功能，因为如果用户无法查看商品详情，就无法完成购买决策，后续购物车、提交订单和支付流程也无法顺利进行。

4.3 用户目标

目标用户：本功能的主要目标用户为消费者（Customer / Buyer）。

用户核心目标

用户进入商品详情页后，希望能够快速完成以下任务：

1. 了解商品基础信息；
2. 查看商品图片；
3. 查看当前销售价格和原价；
4. 选择商品规格，例如颜色、尺寸、型号等；
5. 查看所选SKU的库存；
6. 判断商品是否可以购买；
7. 将商品加入购物车；
8. 直接进入下单流程。

用户使用场景

场景一：用户从商品列表进入商品详情页

用户在商品列表页浏览商品，点击某个商品卡片后进入商品详情页，查看更完整的商品信息。

场景二：用户选择SKU后准备购买

用户在商品详情页选择具体SKU，例如选择“黑色”耳机后，页面展示该SKU对应价格和库存，用户确认后点击“立即购买”。

场景三：用户暂时不购买，先加入购物车

用户对商品感兴趣，但暂时不想立即付款，可以点击“加入购物车”，后续在购物车中统一结算。

场景四:商品不可购买

如果商品已下架或库存为0, 用户进入详情页后应看到明确提示, 并且购买按钮应置灰不可点击。

4.4 页面结构

图4-1 产品详情页线框图

Product Detail Page Wireframe

< 返回

蓝牙耳机

图1 / Product Image

图1

图2

图3

图4

蓝牙耳机Bluetooth Earphones

¥199.00

~~¥299.00~~

132

SKU选择

White

Black

颜色/款式

SKU 15

数量

-

1

+

加入购物车

立即购买

产品描述

蓝牙耳机 / 蓝牙耳机 / 蓝牙耳机

蓝牙耳机

图4-1 商品详情页低保真原型图

4.5 页面字段设计

区域	字段	字段说明	是否必需
商品图片区	商品主图	展示商品主要图片，帮助用户快速识别商品	是
商品图片区	商品图片列表	展示商品多张图片，支持用户查看更多细节	否
商品信息区	商品名称	展示商品名称，例如 Bluetooth Earphones	是
商品信息区	商品ID	用于系统识别商品，前台可不展示	是
商品信息区	商品状态	判断商品是否上架，例如 ON_SALE / OFF_SALE	是
价格区	销售价	用户实际支付的商品价格	是
价格区	原价	用于价格对比展示，帮助用户感知折扣	否
销售数据区	销量	展示商品累计销量，增强用户信任	否
SKU区域	SKU名称	展示规格选项，例如白色、黑色	是
SKU区域	SKU价格	不同SKU可能对应不同价格	是
SKU区域	SKU库存	当前SKU可购买数量	是
库存区域	总库存	商品整体库存，用于整体售罄判断	是
数量区域	购买数量	用户本次计划购买的数量	是
操作区	加入购物车按钮	用户将商品加入购物车	是
操作区	立即购买按钮	用户直接进入订单确认页	是
提示区	错误提示	展示下架、售罄、库存不足等异常信息	是

4.6 核心交互设计

4.6.1 进入商品详情页

用户从商品列表点击商品后，系统根据商品ID请求商品详情接口，并展示商品信息。

如果接口返回成功，则页面展示商品名称、图片、价格、SKU、库存等信息。

如果接口返回失败，则页面展示错误提示。

提示文案：商品信息加载失败，请稍后重试。

4.6.2 SKU选择

如果商品存在多个SKU，用户需要先选择SKU后才能加入购物车或立即购买。

例如：

商品：Bluetooth Earphones

SKU 1: White, 价格 ¥199.00, 库存 10

SKU 2: Black, 价格 ¥209.00, 库存 15

当用户选择不同SKU时，页面需要同步更新：

1. 当前展示价格；
2. 当前SKU库存；
3. 购买按钮状态；
4. 用户可购买数量上限。

如果用户未选择SKU就点击“加入购物车”或“立即购买”，系统应提示用户先选择规格。

提示文案：[请先选择商品规格](#)。

4.6.3 购买数量选择

用户可以通过数量选择器调整购买数量。

购买数量规则：

1. 默认购买数量为1；
2. 购买数量必须大于0；
3. 购买数量不能超过当前SKU库存；
4. 如果当前SKU库存为0，数量选择器不可操作；
5. 如果用户输入非法数量，系统应自动修正或提示错误。

提示文案：[购买数量不能超过当前库存](#)。

4.6.4 加入购物车

当用户点击“加入购物车”时，系统需要校验：

1. 用户是否已登录；
2. 商品是否处于上架状态；
3. 用户是否已选择SKU；
4. 当前SKU是否有库存；
5. 购买数量是否合法。
6. 校验通过后，商品加入购物车。

成功提示：[已加入购物车](#)。

如果用户未登录，可以引导用户先登录。

提示文案：[请先登录后再加入购物车](#)。

4.6.5 立即购买

当用户点击“立即购买”时，系统需要校验：

用户是否已登录；

1. 商品是否处于上架状态；
2. SKU是否已选择；
3. SKU库存是否充足；
4. 购买数量是否合法。
5. 校验通过后，用户进入订单确认页。

如果校验失败，则停留在商品详情页，并展示对应错误提示。

4.7 业务规则

规则一：商品必须处于上架状态才允许购买

当商品状态为 **ON_SALE** 时，用户可以正常选择SKU、加入购物车和立即购买。

当商品状态为 **OFF_SALE** 时，商品不可购买。

页面表现：

1. 显示商品已下架提示；
2. 加入购物车按钮置灰；
3. 立即购买按钮置灰；

用户无法提交购买行为。

提示文案：该商品已下架，暂时无法购买。

规则二：用户必须选择**SKU**后才能购买

如果商品存在SKU列表，用户必须选择一个具体SKU后才能加入购物车或立即购买。

原因是订单需要绑定具体SKU，用于确定价格、库存和后续发货规格。

提示文案：请先选择商品规格。

规则三：页面价格应根据用户选择的**SKU**动态变化

商品可能存在商品基础价格，也可能存在SKU价格。

当用户未选择SKU时，页面可以展示默认商品价格或价格区间。

当用户选择具体SKU后，页面应展示该SKU对应价格。

例如：

用户选择 White SKU：价格显示 ¥199.00 库存显示 10

用户选择 Black SKU：价格显示 ¥209.00 库存显示 15

规则四：库存展示应以当前**SKU**库存为准

当用户选择SKU后，页面应展示当前SKU库存，而不是只展示商品总库存。

原因是不同SKU的库存可能不同。如果只展示商品总库存，用户可能误以为所有规格都有库存，导致下单失败。

规则五：库存为**0**时不可购买

当商品总库存为0，或用户选择的SKU库存为0时，购买按钮应置灰，不允许用户点击。

页面提示：当前商品库存不足。

按钮文案可以改为：**已售罄**

规则六:加入购物车不代表锁定库存

用户将商品加入购物车时, 系统不应立即扣减库存。
原因是加入购物车只是用户的购买意向, 不代表用户一定会支付。
库存应在提交订单或支付阶段根据系统库存策略进行锁定或扣减。

规则七:提交订单前必须重新校验商品状态和库存

商品详情页展示的信息可能存在缓存或延迟, 因此用户点击“立即购买”或从购物车提交订单时, 后端必须重新校验:

- 1. 商品是否上架;
- 2. SKU是否有效;
- 3. 价格是否变化;
- 4. 库存是否充足;
- 5. 购买数量是否合法。

如果后端校验失败, 应阻止用户继续提交订单。

规则八:价格展示需要进行金额格式转换

接口中的价格通常以分为单位返回, 例如:price = 19900
前端页面应展示为:¥199.00
不能直接展示为:¥19900
否则会造成严重的价格展示错误。这个错误一旦上线, 就可以直接让产品经理体验什么叫“公开处刑”。

4.8 异常处理与错误提示

异常场景	触发条件	页面表现	提示文案
商品信息加载失败	商品详情接口请求失败	页面展示加载失败状态	商品信息加载失败, 请稍后重试
商品已下架	商品状态为 OFF_SALE	购买按钮置灰	该商品已下架, 暂时无法购买

商品售罄	商品总库存为0	购买按钮置灰, 显示已售罄	当前商品已售罄
SKU库存不足	用户选择的SKU库存为0	当前SKU不可购买	当前规格库存不足
未选择SKU	用户未选择SKU就点击购买	阻止操作	请先选择商品规格
购买数量超过库存	buyCount > skuStock	阻止操作	购买数量不能超过当前库存
用户未登录	未登录用户点击加入购物车或立即购买	引导登录	请先登录后再操作
价格发生变化	下单前后价格不一致	阻止原价格下单, 提示刷新	商品价格已更新, 请重新确认
网络异常	请求超时或网络失败	保持当前页面, 允许重试	网络异常, 请稍后重试

4.9 验收标准

4.9.1 页面展示验收

1. 用户进入商品详情页后, 可以看到商品名称、商品图片、价格、销量、SKU和库存信息。=
2. 商品图片能够正常加载。
3. 商品销售价展示格式正确, 例如 ¥199.00。
4. 如果存在原价, 页面应展示原价。
5. 如果存在销量, 页面应展示销量。

4.9.2 SKU交互验收

1. 页面可以展示所有可选SKU。
2. 用户选择不同SKU后, 页面价格能够正确变化。
3. 用户选择不同SKU后, 页面库存能够正确变化。

4. 库存为0的SKU不可购买。
5. 用户未选择SKU时点击购买, 系统提示“请先选择商品规格”。

4.9.3 库存规则验收

1. 当商品总库存为0时, 购买按钮置灰。
2. 当当前SKU库存为0时, 购买按钮置灰。
3. 当用户购买数量超过库存时, 系统阻止操作。
4. 页面展示库存数量与接口返回库存一致。
5. 提交订单前, 系统会重新校验库存。

4.9.4 商品状态验收

1. 当商品状态为 ON_SALE 时, 用户可以正常购买。
2. 当商品状态为 OFF_SALE 时, 页面展示“商品已下架”。
3. OFF_SALE 商品的“加入购物车”和“立即购买”按钮不可点击。
4. 如果商品在购物车中被下架, 用户结算时不能提交该商品订单。

4.9.5 加入购物车验收

1. 用户选择SKU和数量后, 可以成功加入购物车。
2. 加入购物车成功后, 页面展示成功提示。
3. 未登录用户点击加入购物车时, 系统引导登录。
4. 商品下架或库存不足时, 不能加入购物车。

4.9.6 立即购买验收

1. 用户选择SKU和数量后, 可以点击立即购买。
2. 点击立即购买后, 系统跳转至订单确认页。
3. 商品下架、库存不足、未选择SKU时, 不能进入订单确认页。
4. 跳转订单确认页前, 系统应校验商品状态、SKU状态、库存和价格。

4.10 埋点与数据记录

虽然题目没有强制要求埋点, 但从产品经理角度看, 商品详情页是转化链路的重要节点, 因此建议记录以下基础数据。

埋点事件	触发时机	用途
product_detail_view	用户进入商品详情页	统计商品曝光和访问量
sku_select	用户选择SKU	分析用户偏好的规格
add_to_cart_click	用户点击加入购物车	分析加购意愿
buy_now_click	用户点击立即购买	分析购买意愿
add_to_cart_success	加入购物车成功	统计加购成功率
buy_now_success	成功进入订单确认页	分析详情页到下单页转化
product_unavailable_show	展示下架或售罄提示	监控不可购买商品对用户体验的影响

这些数据可以帮助后续分析商品详情页的访问量、加购率、购买转化率以及库存异常对转化的影响。

4.11 与其他模块的关系

商品详情页不是孤立功能，它需要与多个模块协同工作。

关联模块	关系说明
商品管理模块	后台维护商品名称、图片、价格、状态等信息，前台商品详情页读取并展示
SKU管理模块	商品详情页展示SKU选项，并根据SKU展示价格和库存
库存管理模块	商品详情页展示库存，下单前需要库存校验
购物车模块	用户点击加入购物车后，将商品、SKU和数量加入购物车
订单模块	用户点击立即购买后，进入订单确认和提交订单流程
用户模块	判断用户是否登录，未登录用户需要先登录
支付模块	商品详情页不直接处理支付，但会引导用户进入后续订单支付流程

4.12 本期MVP范围与后续迭代

本期MVP实现范围

本期商品详情页主要实现以下能力：

1. 展示商品基础信息；
2. 展示商品图片；
3. 展示商品销售价和原价；
4. 展示SKU列表；
5. 支持SKU选择；
6. 根据SKU展示价格和库存；
7. 支持加入购物车；
8. 支持立即购买；
9. 支持商品下架和库存不足提示。

后续可迭代方向

后续版本可以继续增加以下能力：

1. 商品评价展示；
2. 商品详情富文本介绍；
3. 商品参数表；
4. 商品推荐；
5. 优惠券领取；
6. 促销活动标签；
7. 收藏商品；
8. 分享商品；
9. 到货提醒；
10. 相似商品推荐。

MVP阶段暂不建设这些功能，原因是它们不影响基础交易闭环，且会增加前端展示、后台配置和业务规则复杂度。

五、API理解题

本部分基于工程团队提供的商品详情页 API Response 进行分析, 说明前端商品详情页可以展示哪些信息、价格字段应如何处理, 以及在商品上下架、库存变化和 SKU 选择等场景下, 页面应如何响应。同时, 本部分也会从产品经理视角补充说明当前 API 仍然缺少哪些关键字段。

5.1 根据该 API Response, 商品详情页可以展示哪些信息?

根据该 API Response, 商品详情页可以展示商品的基础信息、价格信息、库存信息、销量信息、商品图片以及 SKU 信息。

首先, 页面可以展示商品基础信息。

其中, `productId = 10001` 是商品唯一 ID, 主要用于系统内部识别、接口调用、埋点统计和订单关联, 通常不一定直接展示给普通用户。`productName = "Bluetooth Earphones"` 可以作为商品详情页的商品标题展示。`status = "ON_SALE"` 表示该商品当前处于在售状态, 用户可以正常浏览并进行购买操作。

其次, 页面可以展示商品价格信息。

API 中返回了 `price = 19900` 和 `originalPrice = 29900`。其中 `price` 可以理解为当前销售价, `originalPrice` 可以理解为原价或划线价。页面可以突出展示当前销售价, 同时以较弱样式展示原价, 用于表达折扣或促销信息。

第三, 页面可以展示库存与销量信息。

`stock = 25` 表示该商品当前总库存为 25 件, 页面可以展示为“库存 25 件”, 也可以用于判断购买按钮是否可点击。`salesCount = 132` 表示该商品累计销量为 132, 可以展示为“已售 132 件”, 帮助用户判断商品热度。

第四, 页面可以展示商品图片。

`images` 字段中返回了商品图片地址, 前端可以使用该图片作为商品主图。如果未来返回多张图片, 也可以扩展为商品轮播图或详情图片展示。

第五, 页面可以展示 SKU 规格信息。

`skuList` 字段中包含不同规格的商品, 例如 White 和 Black。每个 SKU 都有独立的 `skuId`、`skuName`、`price` 和 `stock`。因此, 页面可以展示颜色选择项, 并在用户选择不同 SKU 后动态更新价格和库存。

整体来看, 该 API 已经能够支持一个基础商品详情页的核心展示, 包括商品名称、商品图片、当前价格、原价、库存、销量、商品状态以及 SKU 选择。

5.2 price = 19900 应该如何展示给用户 ?

price = 19900 不应该直接展示为“19900”。在电商系统中, 价格字段通常会以“分”为单位返回, 而不是直接以“元”为单位返回。

因此, 前端需要将 19900 除以 100 后展示给用户:

$19900 / 100 = 199.00$

所以页面上应展示为: **¥199.00**

同理, originalPrice = 29900 应展示为: **¥299.00**

在页面展示时, 当前价格可以使用更醒目的样式, 例如加粗或高亮; 原价可以使用灰色划线价展示。一个合理的展示方式是:

当前价格: ¥199.00

原价: ¥299.00

这样既符合用户对价格的认知, 也避免因为单位理解错误导致价格展示异常。否则把一个蓝牙耳机显示成 ¥19900, 用户大概会以为它能顺便替自己找工作, 离谱但不完全没市场。

5.3 如果 status = OFF_SALE, 页面应该如何处理 ?

如果 status = OFF_SALE, 说明该商品已经下架, 不应该允许用户继续购买。

页面可以继续展示商品的基础信息, 但需要在购买区域明确提示: “该商品已下架, 暂时无法购买。”

同时, “加入购物车”和“立即购买”按钮应置灰并禁用, 用户不能点击按钮进入下单流程。这样可以避免用户提交无效订单, 也可以减少后端异常校验压力。

如果用户是从购物车、收藏夹、历史订单或外部链接进入该商品详情页, 页面仍然可以打开, 但必须清楚展示下架状态。这样用户能够理解商品为什么无法购买, 而不是看到一个空白页或系统错误。

此外, 页面也可以提供“查看相似商品”或“返回商品列表”等入口, 引导用户继续浏览其他商品, 减少用户流失。

因此, `OFF_SALE` 状态下的核心处理原则是: 商品可以展示, 但不能购买; 页面必须明确提示商品状态; 所有购买相关操作必须禁用。

5.4 如果商品总库存为 0, 购买按钮应该如何展示?

如果商品总库存 `stock = 0`, 说明该商品当前没有可售库存。此时页面应该将“加入购物车”和“立即购买”按钮置灰并禁用。

按钮文案可以改为:

已售罄

或者:

库存不足

同时, 库存区域应显示明确提示, 例如: “当前商品暂无库存。”

数量选择器也应该禁用, 或者限制用户无法增加购买数量。

需要注意的是, 即使商品状态仍然是 `ON_SALE`, 只要库存为 0, 用户也不应该能够提交订单。商品状态表示商品是否允许售卖, 而库存决定商品当前是否真的可以被购买。

如果后续系统支持补货提醒功能, 可以在该场景下展示“到货提醒”按钮。但对于 MVP 版本来说, 补货提醒不是完成基础交易闭环的必要功能, 可以作为后续优化功能。

5.5 如果用户选择 Black SKU, 页面价格和库存应该如何变化?

如果用户选择 Black SKU, 页面应根据 `skuList` 中 Black 对应的 SKU 信息动态更新价格和库存。

Black SKU 的信息如下:

`skuId = 20002`

`skuName = "Black"`

`price = 20900`

`stock = 15`

因此, 当用户选择 Black SKU 后, 页面展示价格应更新为: **¥209.00**

库存应更新为: **库存 15 件**

同时, 订单提交时也应该传递 Black SKU 对应的 `skuId = 20002`, 而不是只传递商品 ID。因为同一个商品下, 不同 SKU 可能拥有不同价格、库存、颜色或规格。如果订单只记录商品 ID, 就无法准确知道用户购买的是 White 还是 Black, 也无法正确扣减对应 SKU 的库存。

因此, SKU 选择后的页面联动逻辑可以描述为:

用户选择 Black SKU → 页面价格更新为 ¥209.00 → 页面库存更新为 15 件 → 提交订单时传递 skuld = 20002 → 后端基于该 SKU 校验价格和库存。

如果某个 SKU 库存为 0, 该 SKU 应显示为不可购买状态, 用户不能选择该规格继续提交订单。

5.6 该 API Response 还缺少哪些字段?

虽然当前 API 已经能够支持商品详情页的基础展示, 但从产品经理视角来看, 它仍然缺少一些关键字段。建议补充以下字段:

1. 商品详情描述字段: **productDescription**

当前 API 只返回了商品名称和图片, 但没有返回商品详细介绍。商品详情页通常需要展示商品卖点、功能说明、适用场景、材质、尺寸、售后说明等内容。

如果没有商品详情描述, 用户只能看到商品名称和价格, 无法充分了解商品价值, 会影响购买决策。

2. 商品分类字段: **categoryId / categoryName**

API 中缺少商品分类信息。商品分类可以帮助系统判断该商品属于哪个类目, 例如“数码产品”“耳机”“蓝牙设备”等。

该字段不仅可以用于页面展示, 也可以用于商品推荐、搜索筛选、运营管理和数据分析。

3. SKU 属性字段: **skuAttributes**

当前 SKU 只有 **skuName**, 例如 White 和 Black, 但没有明确说明该 SKU 对应的是颜色、尺寸还是其他属性。

更合理的结构应该包括 SKU 属性, 例如:

颜色: Black

尺寸: Standard

版本: Basic

这样前端可以更清晰地展示规格选择项, 也方便后续扩展多规格商品。

4. 商品上下架时间字段: **saleStartTime / saleEndTime**

当前 API 只返回商品状态, 但没有返回商品的上架时间或下架时间。

如果商品涉及定时上架、限时销售、预售或促销活动, 就需要这些时间字段来支持页面展示和业务判断。例如页面可以展示“即将开售”“活动剩余时间”等信息。

5. 运费与配送信息字段: **shippingFee / deliveryInfo**

商品详情页通常需要展示配送方式、预计送达时间、运费规则等信息。当前 API 没有返回这些字段。

对于用户来说, 商品价格只是购买决策的一部分, 运费和配送时间也会影响是否下单。例如同样价格的商品, 一个包邮、一个运费很高, 用户选择可能完全不同。

6. 售后政策字段: **returnPolicy / refundPolicy**

商品详情页还应展示退换货政策、退款规则和售后说明。尤其是电商系统中, 用户在购买前通常会关心是否支持七天无理由退货、是否支持退款、是否有质保等。

如果缺少售后政策, 用户可能会降低购买信任感, 也会增加客服咨询压力。

7. 商品状态展示文案字段: **statusText**

当前 API 只返回了 `status = ON_SALE`, 这是偏技术化的状态值。前端可以自行转换为“在售”, 但如果不同状态对应复杂文案, 最好由后端或配置系统返回对应展示文案。

例如:

ON_SALE: 在售

OFF_SALE: 已下架

SOLD_OUT: 已售罄

COMING_SOON: 即将开售

这样可以减少前端硬编码, 也方便后续统一维护状态文案。

综上, 建议补充的核心字段包括: 商品详情描述、商品分类、SKU 属性、上下架时间、配送信息、售后政策和状态展示文案。这些字段可以帮助商品详情页从“能展示基础商品信息”进一步升级为“能支持用户完整购买决策”的页面。

六、简单代码逻辑阅读题

本部分主要针对题目中给出的伪代码进行业务含义分析。该函数的核心作用是判断用户在当前商品状态、SKU 库存和购买数量条件下，是否可以提交订单。虽然代码本身较简单，但它体现了电商系统中非常重要的后端校验逻辑：前端页面可以引导用户操作，但最终是否允许下单，必须由后端根据商品状态和库存进行判断。

题目中的伪代码如下：

```
function canSubmitOrder(productStatus, skuStock, buyCount) {  
  if (productStatus !== 'ON_SALE') {  
    return {  
      canSubmit: false,  
      reason: 'Product is off sale'  
    };  
  }  
  if (skuStock <= 0) {  
    return {  
      canSubmit: false,  
      reason: 'Insufficient inventory'  
    };  
  }  
  if (buyCount > skuStock) {  
    return {  
      canSubmit: false,  
      reason: 'Purchase quantity exceeds available inventory'  
    };  
  }  
  return {  
    canSubmit: true,  
    reason: ''  
  };  
}
```


6.1 这个函数主要用于判断什么？

这个函数主要用于判断用户是否可以提交订单。

具体来说, 它会根据三个关键条件进行判断:

第一, 商品是否处于在售状态。

如果商品不是 `ON_SALE`, 说明商品可能已经下架、暂停销售或不可购买, 此时用户不能提交订单。

第二, 所选 SKU 是否有库存。

如果 `skuStock <= 0`, 说明该 SKU 当前没有可售库存, 用户不能提交订单。

第三, 用户购买数量是否超过当前 SKU 库存。

如果用户想购买的数量 `buyCount` 大于当前 SKU 库存 `skuStock`, 说明库存不足以满足该订单, 用户也不能提交订单。

只有当商品处于在售状态、SKU 有库存, 并且购买数量不超过库存时, 系统才允许用户提交订单。

从产品角度来看, 这个函数可以被理解为订单提交前的基础校验规则。它的作用是避免用户购买已经下架的商品、无库存商品, 或者购买数量超过库存的商品。

6.2 在什么情况下用户不能提交订单？

根据该函数逻辑, 用户在以下三种情况下不能提交订单。

1. 商品不是在售状态

当 `productStatus !== 'ON_SALE'` 时, 函数会返回:

```
{
  canSubmit: false,
  reason: 'Product is off sale'
}
```

这表示商品当前不是在售状态, 用户不能提交订单。

对应的业务场景包括:

- 商品已下架;
- 商品被运营人员临时关闭;
- 商品还未正式上架;
- 商品因为异常原因被平台禁售。

在这些情况下，即使用户已经把商品加入购物车，也不能继续提交订单。系统应提示用户：

“商品已下架，无法购买。”

2. SKU 库存小于或等于 0

当 `skuStock <= 0` 时，函数会返回：

```
{  
  canSubmit: false,  
  reason: 'Insufficient inventory'  
}
```

这表示用户选择的 SKU 当前没有库存，不能提交订单。

例如，商品整体可能仍然在售，但用户选择的某个颜色或规格已经卖完。比如白色耳机还有库存，但黑色耳机库存为 0，那么用户选择黑色 SKU 时就不能下单。

页面应提示用户：“库存不足，请选择其他规格。”

3. 用户购买数量超过 SKU 可用库存

当 `buyCount > skuStock` 时，函数会返回：

```
{  
  canSubmit: false,  
  reason: 'Purchase quantity exceeds available inventory'  
}
```

这表示用户想购买的数量超过了当前可售库存。

例如，当前 SKU 库存为 3 件，但用户想购买 5 件。此时系统不能让用户提交订单，因为库存无法满足订单需求。

页面应提示用户：

“购买数量超过可用库存，请修改购买数量。”

6.3 如果 `productStatus = 'ON_SALE'`，`skuStock = 3`，`buyCount = 5`，函数应该返回什么结果？

在这个案例中：

`productStatus = 'ON_SALE'`

`skuStock = 3`

`buyCount = 5`

首先, 商品状态是 `ON_SALE`, 说明商品处于在售状态, 因此可以通过第一层校验。

其次, `skuStock = 3`, 库存大于 0, 因此可以通过第二层校验。

但是, `buyCount = 5`, 用户想购买 5 件, 而当前 SKU 库存只有 3 件。由于购买数量超过了库存, 即:

`buyCount > skuStock`

所以函数会进入第三个判断条件, 并返回不能提交订单的结果。

函数应返回:

```
{
  canSubmit: false,
  reason: 'Purchase quantity exceeds available inventory'
}
```

对应的业务含义是:

商品虽然在售, 并且当前 SKU 也有库存, 但用户购买数量超过了可用库存, 因此不能提交订单。

系统应提示用户减少购买数量, 例如:

“当前库存仅剩 3 件, 请修改购买数量后再提交订单。”

这个场景非常典型, 因为用户看到“有库存”不代表可以买任意数量。库存只有 3 件却想买 5 件, 系统当然要拦下来, 不然仓库同事只能现场表演无中生有, 人类商业文明又多一个悲剧。

6.4 还应该补充哪些额外校验？

当前函数只校验了商品状态、SKU 库存和购买数量, 能够覆盖最基础的下单限制。但在真实电商系统中, 提交订单还需要补充更多业务校验, 才能避免异常订单和交易风险。建议补充以下校验:

1. 用户登录状态校验

在提交订单前, 系统需要判断用户是否已经登录。

如果用户未登录, 应阻止提交订单, 并引导用户先登录。因为订单需要绑定用户 ID, 用于后续查看订单、支付、退款、售后和物流查询。

对应规则可以是:

如果用户未登录, 则不能提交订单。

提示文案: “请先登录后再提交订单。”

2. 收货地址校验

用户提交订单前必须选择有效的收货地址。

如果用户没有填写收货人、手机号、详细地址，或者地址不在配送范围内，系统应阻止提交订单。

对应规则可以是：

如果收货地址为空或无效，则不能提交订单。

提示文案：“[请填写有效的收货地址。](#)”

3. 购买数量合法性校验

当前代码只判断 `buyCount > skuStock`，但还应该判断购买数量是否为合法数字。

例如：

购买数量不能小于 1；

购买数量不能为 0；

购买数量不能为负数；

购买数量不能是小数；

购买数量不能是非数字字符。

否则用户可能提交 `buyCount = 0` 或 `buyCount = -1` 这种异常数据。虽然正常用户不会这么干，但总有人类和脚本喜欢挑战系统边界，产品和技术只能被迫成熟。

对应规则可以是：

如果购买数量小于 1 或不是正整数，则不能提交订单。

提示文案：“[请输入有效的购买数量。](#)”

4. 单人限购校验

部分商品可能存在限购规则，例如每个用户最多购买 2 件。

因此，系统需要判断用户本次购买数量加上历史已购买数量，是否超过限购上限。

对应规则可以是：

如果用户购买数量超过限购数量，则不能提交订单。

提示文案：“[该商品每人限购 2 件。](#)”

该规则在促销商品、稀缺商品和平台补贴商品中非常常见，可以防止恶意囤货或刷单。

5. 商品价格校验

用户提交订单时，系统应重新校验商品价格，不能完全相信前端传来的价格。

因为商品价格可能在用户浏览详情页后发生变化,也可能被前端篡改。因此,后端应以数据库中的实时价格为准。

对应规则可以是:

如果前端传入价格与后端当前价格不一致,应以后端价格为准,必要时提示用户价格已变化。

提示文案:“商品价格已更新,请确认后重新提交订单。”

6. 商品是否仍存在校验

除了判断商品是否在售,还需要判断商品本身是否仍然存在。

例如,商品可能已被运营人员删除,或者商品 ID 无效。此时系统应阻止提交订单。

对应规则可以是:如果商品不存在,则不能提交订单。

提示文案:“商品不存在或已失效。”

7. SKU 是否有效校验

用户提交订单时,系统需要判断所选 SKU 是否仍然属于该商品,并且 SKU 是否有效。

例如,用户选择的 SKU 可能已经被删除,或者前端传入了错误的 `skuId`。如果不校验,可能会导致订单商品规格错误,甚至影响库存扣减。

对应规则可以是:

如果 SKU 不存在或不属于当前商品,则不能提交订单。

提示文案:“商品规格无效,请重新选择。”

8. 购物车商品有效性校验

如果用户是从购物车提交订单,系统还需要检查购物车中的商品是否仍然有效。

可能出现的异常包括:

商品已下架;

- SKU 已失效;
- 库存不足;
- 价格已变化;
- 商品已被删除。

对应规则可以是:

如果购物车商品已失效,则不能提交订单。

提示文案:“购物车中存在失效商品,请修改后再提交。”

9. 支付方式校验

如果提交订单时需要选择支付方式,系统应判断支付方式是否可用。

例如，某些支付方式可能临时维护，或者当前订单金额不支持某种支付方式。

对应规则可以是：

如果支付方式不可用，则不能提交订单。

提示文案：“当前支付方式不可用，请选择其他支付方式。”

10. 风控校验

在真实电商系统中，提交订单还可能需要进行基础风控判断。

例如：

- 用户账号是否异常；
- 是否存在频繁下单行为；
- 是否存在恶意刷单风险；
- 收货地址或手机号是否异常；
- 订单金额是否异常。

如果触发风控规则，系统可以限制提交订单，或要求进一步验证。

对应规则可以是：

如果订单触发风险控制规则，则暂时不能提交订单。

提示文案：“订单存在异常，请稍后重试或联系客服。”

6.5 总结

总体来看，该函数的业务含义是：在用户提交订单前，系统需要校验商品是否在售、SKU 是否有库存，以及购买数量是否超过库存。只有全部条件满足时，用户才可以提交订单。

当前函数覆盖了最基础的库存和商品状态校验，但真实电商系统还需要补充用户登录、收货地址、购买数量合法性、限购规则、价格变更、SKU 有效性、购物车商品状态、支付方式和风控等校验。

产品经理需要理解每个判断条件背后的业务意义，并能够将代码逻辑转化为清晰的页面提示、交互规则和异常处理方案。

七、订单状态理解题

本部分主要分析电商系统中的订单状态设计。订单状态是电商交易流程中的核心字段之一，它决定了用户、运营人员和系统在不同阶段可以执行哪些操作。例如，用户提交订单后是否可以支付，支付后是否可以申请退款，商家是否可以发货，以及订单最终是否完成。

题目中给出的订单状态包括：

- **PENDING_PAYMENT**: 待支付
- **PAID**: 已支付
- **SHIPPED**: 已发货
- **COMPLETED**: 已完成
- **CANCELLED**: 已取消
- **REFUNDING**: 退款中
- **REFUNDED**: 已退款

这些状态覆盖了一个基础电商系统中的主要订单生命周期，包括正常交易流程和异常退款流程。

7.1 客户提交订单后的初始状态应该是什么？

客户提交订单后，订单的初始状态应该是：

PENDING_PAYMENT: 待支付

原因是用户提交订单只代表订单已经创建成功，但并不代表交易已经完成。此时用户还没有完成付款，系统不能将订单直接标记为“已支付”或“已完成”。

在这个阶段，系统应该生成订单号，并锁定对应商品库存，等待用户在规定时间内完成支付。

例如，用户点击“提交订单”后，系统可以生成一条订单记录，状态为 **PENDING_PAYMENT**。页面上可以显示：

“订单已提交，请在 15 分钟内完成支付。”

该状态下，用户通常可以进行以下操作：

- 继续支付；
- 取消订单；
- 返回订单列表；
- 查看订单详情。

同时，系统也需要设置支付倒计时。如果用户在规定时间内未完成支付，订单应自动取消，并释放被锁定的库存。

因此，提交订单后的初始状态应为 **PENDING_PAYMENT**，因为订单已经创建，但还没有完成付款。

7.2 支付成功后订单状态应该变成什么？

支付成功后，订单状态应该从：

PENDING_PAYMENT：待支付

变为：

PAID：已支付

原因是用户已经完成付款，系统确认收到了支付结果，此时订单进入待发货阶段。

在这个阶段，用户已经不能随意取消订单，运营人员或商家需要在后台看到该订单，并安排后续发货流程。

状态变化可以描述为：

用户提交订单 → 订单状态为 **PENDING_PAYMENT**

用户完成支付 → 支付系统返回成功结果 → 订单状态更新为 **PAID**

当订单状态变为 **PAID** 后，系统可以触发以下操作：

1. 通知用户支付成功；
2. 通知运营人员处理订单；
3. 生成待发货任务；
4. 更新订单支付时间；
5. 记录支付流水号；
6. 准备进入发货流程。

页面可以显示：

“支付成功，等待商家发货。”

需要注意的是，订单状态更新必须以后端支付回调结果为准，而不是只以前端页面显示为准。因为用户看到支付成功页面，不一定代表后端真的收到支付成功通知。电商系统不能靠“用户说他付了”来发货，不然平台很快就会变成公益组织，还是亏到破产那种。

7.3 商家发货后订单状态应该变成什么？

商家发货后，订单状态应该从：

PAID：已支付

变为：

SHIPPED：已发货

原因是订单已经完成支付，并且商家已经进行了发货操作。此时订单进入物流配送阶段。

状态变化可以描述为：

订单支付成功 → 状态为 **PAID**

运营人员填写物流信息并确认发货 → 状态更新为 **SHIPPED**

在 **SHIPPED** 状态下，系统通常需要记录以下信息：

1. 发货时间；
2. 物流公司；
3. 物流单号；
4. 配送状态；
5. 收货地址；
6. 预计送达时间。

用户端页面可以显示：

“商家已发货，商品正在配送中。”

并提供物流查看入口，例如：“[查看物流](#)”

对于运营人员来说，SHIPPED 状态表示该订单已经完成发货处理，不应再出现在“待发货订单”列表中，而应进入“已发货订单”或“配送中订单”列表。

7.4 如果客户在支付时限内没有付款，订单状态应该变成什么？

如果客户在支付时限内没有完成付款，订单状态应该从：

PENDING_PAYMENT：待支付

变为：

CANCELLED：已取消

原因是订单虽然已经创建，但用户没有在规定时间内支付。为了避免无效订单长期占用库存，系统需要自动取消订单，并释放被锁定的库存。

例如，系统设置支付有效时间为 15 分钟。如果用户提交订单后 15 分钟内没有支付，系统应自动执行以下操作：

将订单状态更新为 **CANCELLED**；

释放该订单占用的库存；

记录取消原因，例如“支付超时自动取消”；

在订单详情页展示取消状态；

必要时通知用户订单已取消。

页面可以显示：

“订单已因支付超时自动取消。”

需要注意的是，支付超时取消与用户主动取消都可以进入 **CANCELLED** 状态，但取消原因应该区分记录。例如：

用户主动取消：**User cancelled**

支付超时取消：**Payment timeout**

后台取消：**Admin cancelled**

这样后续进行数据分析时，可以知道订单取消的主要原因，而不是只看到一堆冷冰冰的 **CANCELLED**，像一片产品经理的墓碑。

7.5 哪些状态下客户应该允许申请退款？

在基础电商系统中，客户是否可以申请退款，需要根据订单当前状态和业务规则决定。

一般来说，以下状态可以允许客户申请退款：

1. PAID: 已支付，未发货

当订单已经支付但商家还没有发货时，用户通常应该可以申请退款。

这是最简单的退款场景，因为商品还没有发出，不涉及物流拦截或退货处理。系统可以将订单状态从 **PAID** 更新为 **REFUNDING**，退款成功后再更新为 **REFUNDED**。

状态流转可以是：

PAID → **REFUNDING** → **REFUNDED**

页面可以提示：

“退款申请已提交，请等待处理。”

2. SHIPPED: 已发货

当订单已经发货后, 用户是否可以申请退款, 需要根据平台规则决定。

在很多电商场景中, 已发货订单仍然可以申请退款或退货退款, 但流程会更复杂。因为商品已经进入物流阶段, 可能需要用户拒收、退货, 或者等待商家审核。

状态流转可以是:

SHIPPED → REFUNDING → REFUNDED

但这种情况下, 系统可能需要增加退货物流信息、商家审核状态和退款原因等字段。

3. COMPLETED: 已完成

订单完成后是否允许退款, 也取决于售后政策。

如果平台支持售后退款, 例如 7 天无理由退货, 那么 COMPLETED 状态下也可以允许用户申请退款或售后。

状态流转可以是:

COMPLETED → REFUNDING → REFUNDED

但需要满足一定条件, 例如:

- 仍在售后期限内;
- 商品支持退换货;
- 用户提供退款原因;
- 商品未影响二次销售;
- 商家审核通过。

4. 不建议允许退款的状态

以下状态通常不应允许用户再次申请退款:

PENDING_PAYMENT: 待支付

因为用户还没有付款, 不存在退款问题。如果用户不想买, 可以取消订单。

CANCELLED: 已取消

订单已经取消, 如果未支付则不需要退款; 如果已支付后取消, 则应进入退款流程, 而不是停留在普通取消状态。

REFUNDING: 退款中

订单已经在退款处理中, 不应重复提交退款申请。

REFUNDED: 已退款

退款已经完成, 不应再次申请退款。

因此, 一个较合理的设计是:

允许申请退款的状态: PAID、SHIPPED、COMPLETED

不允许申请退款的状态: PENDING_PAYMENT、CANCELLED、REFUNDING、REFUNDED

其中, PAID 状态下的退款最简单; SHIPPED 和 COMPLETED 状态下的退款需要结合退货、售后和审核规则进行判断。

7.6 订单状态流转流程描述

一个基础电商系统的订单状态流转可以分为正常交易流程、支付超时取消流程和退款流程。

1. 正常交易流程

正常情况下, 订单会按照以下路径流转:

PENDING_PAYMENT → PAID → SHIPPED → COMPLETED

具体含义如下:

用户提交订单后, 订单进入 PENDING_PAYMENT 状态, 等待用户支付。

用户支付成功后, 订单进入 PAID 状态, 等待商家发货。

商家确认发货后, 订单进入 SHIPPED 状态, 等待用户收货。

用户确认收货或系统自动确认收货后, 订单进入 COMPLETED 状态, 交易完成。

这个流程是电商系统最核心的交易闭环。

2. 支付超时或用户取消流程

如果用户提交订单后没有完成支付, 订单可以从 PENDING_PAYMENT 状态变为 CANCELLED。

状态流转为:

PENDING_PAYMENT → CANCELLED

触发场景包括:

- 用户主动取消订单;
- 用户超过支付时限未付款;
- 系统检测到订单异常并取消;
- 运营人员在后台取消订单。

在这个流程中, 如果订单还没有支付, 取消后应释放库存, 但不需要退款。

3. 退款流程

如果用户已经支付订单，则可以根据订单状态进入退款流程。

常见退款状态流转为：

PAID → REFUNDING → REFUNDED

适用于已支付但未发货的订单。

如果订单已经发货，也可能进入退款流程：

SHIPPED → REFUNDING → REFUNDED

适用于用户申请退货退款、拒收或商家同意退款的场景。

如果订单已经完成，但仍在售后期内，也可能进入退款流程：

COMPLETED → REFUNDING → REFUNDED

适用于平台支持售后退款的场景。

其中：

REFUNDING 表示退款申请已提交，正在等待审核或退款处理。

REFUNDED 表示退款已经完成，订单金额已退回用户账户。

7.7 订单状态流转图

订单状态流转图

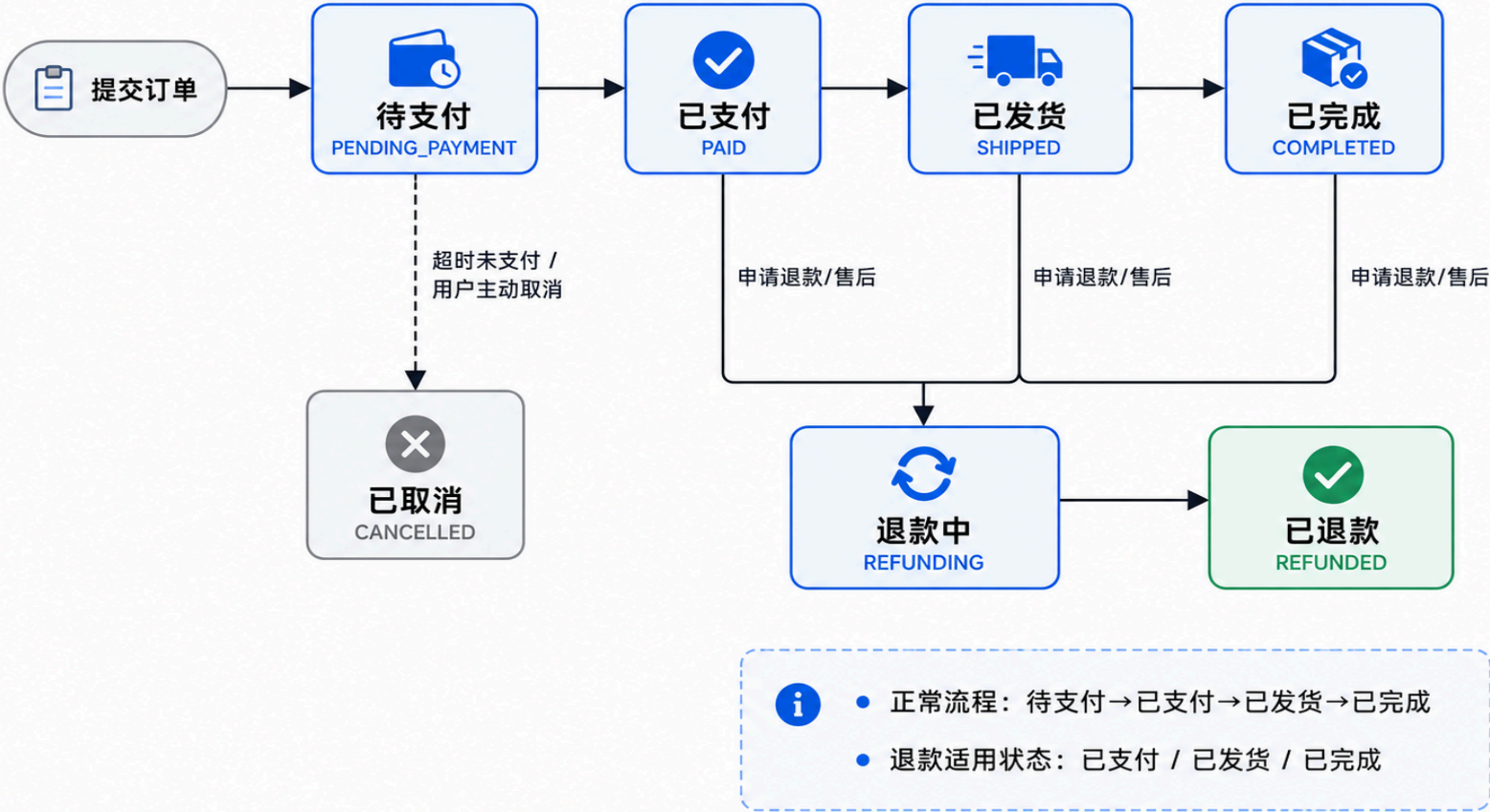


图7-6 订单状态流转图

7.8 订单状态设计中的注意事项

在设计订单状态时，需要注意以下几点：

第一，订单状态应该互斥且清晰。

同一时间，一个订单应该只有一个主状态。例如，一个订单不能同时是 PAID 和 CANCELLED。否则运营后台和用户前端都会混乱，最后没人知道订单到底是活着、死了，还是在交易系统里当薛定谔的订单。

第二, 状态变化应该由明确事件触发。

例如, 支付成功触发 `PENDING_PAYMENT` → `PAID`, 商家发货触发 `PAID` → `SHIPPED`, 用户确认收货触发 `SHIPPED` → `COMPLETED`。

第三, 关键状态变化需要记录时间。

例如:

- 订单创建时间;
- 支付时间;
- 发货时间;
- 完成时间;
- 取消时间;
- 退款申请时间;
- 退款完成时间。

这些时间字段不仅用于页面展示, 也用于运营分析、用户投诉处理和财务对账。

第四, 异常状态需要记录原因。

例如, 订单取消时应记录取消原因; 退款时应记录退款原因; 退款失败时也应记录失败原因。这样可以帮助产品和运营分析问题来源。

第五, 前端展示状态和后端真实状态必须保持一致。

订单详情页、订单列表页、运营后台和支付系统之间需要同步订单状态。如果前端仍显示“待支付”, 但后端已经取消订单, 用户体验会非常糟糕。

7.9 总结

综上, 客户提交订单后的初始状态应为 `PENDING_PAYMENT`; 支付成功后状态变为 `PAID`; 商家发货后状态变为 `SHIPPED`; 用户确认收货或系统自动确认收货后状态变为 `COMPLETED`; 如果用户支付超时或主动取消未支付订单, 状态应变为 `CANCELLED`。

退款方面, 通常可以允许 `PAID`、`SHIPPED` 和 `COMPLETED` 状态下的订单申请退款, 但具体是否允许还需要结合平台售后规则判断。退款流程通常为 `REFUNDING` → `REFUNDED`。

订单状态设计的核心目标是让交易流程清晰、可追踪、可控制。对于产品经理来说，理解订单状态不仅是理解业务流程，也是理解系统规则、异常处理和用户体验的基础。

八、Bug与异常问题分析

本部分主要分析 QA 团队反馈的库存异常问题：

用户在商品详情页看到商品库存为 1，但在提交订单时，系统提示 “Insufficient inventory”。

从表面上看，这似乎是前端页面展示的库存与下单时系统校验的库存不一致。但在真实电商系统中，这类问题可能由缓存、并发购买、SKU 库存不一致、购物车商品状态变化、接口数据不同步、后端库存校验等多种原因导致。

因此，分析该问题时不能只判断“前端显示错了”或“后端接口错了”，而应该从用户操作路径、前后端接口、缓存机制、SKU 库存、订单提交逻辑和数据库库存记录等多个角度进行排查。

8.1 可能原因一：商品详情页库存存在缓存，展示的不是实时库存

商品详情页通常访问量较高，为了提升页面加载速度，系统可能会对商品详情信息进行缓存。例如商品名称、价格、图片、销量和库存等信息可能被缓存在 Redis、CDN 或前端本地缓存中。

如果商品详情页展示的是缓存中的库存，而不是数据库中的实时库存，就可能出现以下情况：

用户打开商品详情页时，页面显示库存为 1；

但实际上数据库中的库存已经被其他订单扣减为 0；

用户提交订单时，后端使用实时库存进行校验；

后端发现库存不足，于是返回 “Insufficient inventory”。

这种情况下，商品详情页显示的库存并不是最新数据，下单接口返回库存不足是合理的。

排查方法

1. 需要确认商品详情页库存字段的数据来源。例如检查商品详情页调用的接口是否直接读取数据库，还是读取缓存数据。
2. 需要检查缓存更新机制。例如库存发生变化后，缓存是否会及时更新或失效。尤其是在库存从 1 变成 0 时，缓存是否仍然保留旧值。
3. 需要查看问题发生时的库存变更记录。如果数据库库存已经是 0，但缓存仍然是 1，就可以判断是缓存未及时刷新导致的展示不一致。

解决建议

对于商品库存这种高实时性字段，建议不要完全依赖长时间缓存。可以采用以下方式优化：

- 商品基础信息可以缓存，但库存信息尽量实时查询；
 - 如果库存需要缓存，应设置较短过期时间；
 - 库存扣减后及时更新或删除缓存；
 - 在商品详情页展示提示，例如“[库存以提交订单时校验结果为准](#)”。
-

8.2 可能原因二：用户浏览商品后，其他用户已经抢先下单

电商系统中，库存是共享资源。用户看到库存为 1 时，并不代表这个库存会一直为他保留。除非系统在用户进入详情页时就锁定库存，但大多数电商系统不会这样做，因为这会导致库存被大量无效浏览占用。

可能的情况是：

- 用户 A 打开商品详情页，看到库存为 1；
- 用户 B 在用户 A 提交订单前购买了最后 1 件；
- 用户 B 的订单成功扣减库存；
- 用户 A 再提交订单时，库存已经变成 0；
- 系统提示用户 A “[库存不足](#)”。

这种情况不是系统 bug，而是正常的并发购买场景。尤其是当商品库存只剩 1 件时，多个用户同时购买很容易发生库存竞争。

排查方法

需要查看该商品或 SKU 在问题发生时间段内的订单记录和库存流水。

重点检查：

- 用户看到库存为 1 的时间；
- 用户提交订单的时间；
- 在这两个时间之间，是否有其他用户成功提交订单或完成支付；
- 该 SKU 的库存是否在这段时间内从 1 变成 0；
- 库存扣减发生在提交订单阶段，还是支付成功阶段。

如果在用户提交订单前，已有其他订单成功占用了最后库存，则该问题属于正常业务逻辑，不是系统错误。

解决建议

前端可以在库存较低时展示提示，例如：

[“库存紧张，请尽快下单。”](#)

同时, 在提交订单失败后, 页面应给出更友好的提示:

“商品库存已发生变化, 请重新选择商品或刷新页面。”

而不是只展示冰冷的 “Insufficient inventory”。用户不是后端日志阅读器, 虽然有时候很希望他们是。

8.3 可能原因三: SKU 库存和商品总库存不一致

商品详情页可能展示的是商品总库存 `product stock`, 但提交订单时校验的是用户选择的具体 SKU 库存 `sku stock`。

例如:

- 商品总库存显示为 1;
- 但用户选择的 Black SKU 库存为 0;
- 另一个 White SKU 库存为 1;
- 页面如果没有正确根据用户选择的 SKU 更新库存, 就可能仍然显示商品总库存 1;
- 提交订单时, 后端根据用户选择的 Black SKU 校验库存, 发现库存为 0;
- 于是提示库存不足。

这种问题在多规格商品中非常常见。商品总库存和 SKU 库存必须保持一致, 否则用户就会看到“商品有库存”, 但自己选中的规格其实没库存。

排查方法

需要检查商品详情页库存展示逻辑和订单提交参数。

重点查看:

1. 商品详情页展示的库存字段是商品总库存, 还是当前选中 SKU 的库存;
2. 用户选择 SKU 后, 页面是否动态更新库存;
3. 提交订单时传递的是 `productId` 还是 `skuId`;
4. 后端实际校验的是商品总库存还是 SKU 库存;
5. 商品总库存是否等于所有 SKU 库存之和。

如果页面展示的是总库存, 但提交订单校验的是 SKU 库存, 就需要进一步确认 SKU 层面的库存是否为 0。

解决建议

商品详情页应优先展示用户当前选中 SKU 的库存, 而不是只展示商品总库存。

交互逻辑可以设计为:

- 用户未选择 SKU 时, 展示商品总库存或提示“[请选择规格](#)”;

- 用户选择具体 SKU 后, 库存展示切换为该 SKU 的库存;
- 如果该 SKU 库存为 0, 则禁用购买按钮;
- 提交订单时必须传递 `skuld`, 后端根据 `skuld` 进行库存校验和扣减。

这样可以减少用户误解, 也能避免商品总库存与 SKU 库存展示不一致的问题。

8.4 可能原因四:购物车中的商品已经失效或库存发生变化

如果用户是从购物车提交订单, 问题也可能来自购物车数据没有及时更新。

例如:

- 用户之前将商品加入购物车时, 库存为 1;
- 之后商品库存被其他订单扣减为 0;
- 购物车页面仍然显示旧库存或未刷新商品状态;
- 用户直接从购物车提交订单;
- 后端重新校验库存时发现库存不足;
- 于是返回 “Insufficient inventory”。

此外, 购物车中的商品也可能出现以下异常:

- 商品已下架;
- SKU 已删除;
- 商品价格已变化;
- SKU 库存变为 0;
- 商品被运营人员修改或禁售。

如果购物车页面没有及时校验商品有效性, 就会在提交订单阶段暴露问题。

排查方法

需要确认用户的操作入口是商品详情页直接购买, 还是购物车提交订单。

如果是购物车, 需要检查:

1. 购物车页面是否重新拉取商品最新状态;
2. 购物车中的商品库存是否实时更新;
3. 购物车是否标记失效商品;
4. 提交订单前是否进行商品有效性校验;
5. 该商品加入购物车时间与提交订单时间间隔是否较长。

如果用户很久之前加入购物车, 之后直接提交订单, 则库存变化导致提交失败是合理情况。

解决建议

购物车页面应在用户进入页面或勾选商品时重新校验商品状态。

如果商品已失效，应展示：“商品已下架，无法购买。”

如果库存不足，应展示：“库存不足，请修改购买数量或删除商品。”

如果价格发生变化，应展示：“商品价格已变化，请确认后再提交订单。”

这样可以将问题提前暴露在购物车阶段，而不是等用户点击提交订单后才失败。

8.5 可能原因五：订单提交 API 重新校验库存，因此与前端展示结果不同

在电商系统中，前端展示库存只能作为用户参考，不能作为最终判断依据。真正决定订单是否可以提交的，应该是后端订单提交 API 的库存校验结果。

原因很简单：前端数据可能过期，也可能被篡改。如果系统只相信前端库存展示，用户甚至可以通过修改请求参数提交异常订单。人类写脚本钻漏洞的热情，有时比写求职信还高。

因此，订单提交 API 必须重新校验：

1. 商品是否存在；
2. 商品是否在售；
3. SKU 是否有效；
4. SKU 是否有库存；
5. 购买数量是否超过库存；
6. 库存是否已被其他订单锁定或扣减。

如果订单提交 API 重新校验库存后发现库存不足，就应该拒绝提交订单。

排查方法

需要检查订单提交 API 的日志和返回结果。

重点查看：

- 订单提交 API 收到的 `productId` 和 `skuId` 是否正确；
- 提交的购买数量 `buyCount` 是否正确；
- 后端查询到的实时库存是多少；
- 是否存在库存锁定记录；
- 是否有其他订单占用了该库存；
- 后端返回 “`Insufficient inventory`” 的具体判断条件是哪一条。

如果后端实时库存确实为 0，那么订单提交 API 拒绝下单是正确行为。问题重点应转向前端库存展示为什么没有同步更新。

8.6 可能原因六:前端展示库存和后端实际库存不同步

前端页面可能在用户停留期间没有自动刷新库存。用户打开商品详情页时库存为 1, 但页面停留了一段时间后, 库存已经被其他用户购买。由于前端没有刷新, 用户仍然看到旧库存。

此外, 也可能存在以下问题:

- 前端使用了旧接口数据;
- 前端没有监听 SKU 变化;
- 前端本地状态没有更新;
- 页面返回时没有重新请求商品详情接口;
- 库存字段展示错误;
- 前端展示了商品总库存, 而不是 SKU 库存。

这些都可能导致前端显示与后端实际库存不一致。

排查方法

可以通过复现用户路径来检查问题:

1. 打开商品详情页;
2. 记录接口返回的库存字段;
3. 停留一段时间;
4. 让另一个用户购买该商品;
5. 观察原页面是否更新库存;
6. 再提交订单, 看是否出现库存不足。

同时, 需要使用浏览器开发者工具检查:

- 商品详情接口返回的库存是多少;
- 前端页面展示的库存是否与接口一致;
- 用户切换 SKU 后是否重新渲染库存;
- 提交订单接口传递的 SKU 和数量是否正确。

如果接口返回库存已经是 0, 但页面仍展示 1, 则可能是前端状态更新或页面渲染问题。

如果接口返回库存仍是 1, 但数据库实际库存是 0, 则可能是后端接口或缓存问题。

8.7 建议的排查步骤

针对该问题, 可以按照以下步骤进行排查:

第一步: 确认用户操作路径

首先需要确认用户是从哪里进入下单流程的:

1. 商品详情页点击“立即购买”;
2. 购物车勾选商品后提交订单;
3. 历史页面返回后再次提交;
4. 活动页或外部链接进入商品页。

不同路径对应的问题原因不同。如果是购物车路径, 需要重点检查购物车商品状态; 如果是商品详情页路径, 需要重点检查详情页库存展示和 SKU 联动。

第二步: 确认用户选择的 **SKU** 和购买数量

需要查看用户提交订单时的请求参数, 包括:

- `productId`;
- `skuId`;
- `buyCount`;
- 用户 ID;
- 提交时间;
- 前端页面展示库存。

如果用户选择的 SKU 库存为 0, 但商品总库存为 1, 则问题很可能是 SKU 库存展示错误。

第三步: 检查商品详情页接口返回值

查看用户打开商品详情页时, 商品详情接口实际返回的库存值。

需要确认:

1. 接口返回的 `stock` 是商品总库存还是 SKU 库存;
2. `skuList` 中各 SKU 的库存是多少;
3. 前端展示的库存是否与接口返回一致;
4. 接口是否读取缓存。

第四步: 检查订单提交 **API** 的库存校验结果

查看订单提交 API 日志, 确认后端为什么返回 “`Insufficient inventory`”。

需要确认:

1. 后端查询到的实时库存是多少；
2. 库存是否已被其他订单锁定；
3. 购买数量是否超过库存；
4. 商品或 SKU 是否已失效；
5. 是否有并发下单导致库存被抢占。

第五步:检查库存流水和订单记录

查看该 SKU 在问题发生时间段内的库存流水和订单记录。

重点确认：

1. 库存从 1 变成 0 的时间；
2. 是谁触发了库存扣减；
3. 库存扣减对应哪一笔订单；
4. 用户提交订单前是否已有其他订单占用最后库存；
5. 是否存在库存回滚失败或重复扣减。

库存流水是排查这类问题的关键证据。如果没有库存流水，排查会很痛苦，像在没有监控的系统里找 bug，堪称产品和研发共同修行。

第六步:判断问题类型

最后根据证据判断问题属于哪一类：

- 如果数据库库存已为 0，但详情页仍显示 1，可能是缓存或前端未刷新问题。
- 如果商品总库存为 1，但用户选择的 SKU 库存为 0，可能是 SKU 库存展示逻辑问题。
- 如果用户打开页面后，其他用户购买了最后库存，属于正常并发场景。
- 如果后端库存校验错误，则需要研发修复库存判断逻辑。
- 如果购物车商品状态未更新，则需要优化购物车有效性校验。

8.8 产品层面的优化建议

针对该问题，除了技术排查，也可以从产品设计层面进行优化。

1. 提交订单前重新拉取库存

在用户点击“提交订单”前，前端可以调用接口重新确认商品库存、商品状态和 SKU 状态。如果发现库存不足，应提前提示用户，而不是进入订单提交后才报错。

2. 商品详情页展示低库存提示

当库存较低时，例如库存小于等于 5，可以展示提示：“库存紧张，请尽快下单。”

这样可以let用户意识到库存可能随时变化，降低用户对库存固定性的误解。

3. 优化错误提示文案

不要只展示 “Insufficient inventory”，可以改成更用户友好的文案：

“商品库存已发生变化，请重新选择商品或刷新页面。”

如果知道具体原因，也可以进一步提示：

“当前规格已售罄，请选择其他规格。”

4. 区分商品总库存和 SKU 库存

页面应明确根据用户选择的 SKU 展示库存。如果用户还没有选择 SKU，可以提示：

“请选择商品规格。”

用户选择 SKU 后，再展示该 SKU 的库存。这样可以减少商品总库存与 SKU 库存混淆。

5. 购物车商品状态实时校验

购物车页面应在用户进入页面、勾选商品、修改数量和提交订单前进行商品状态校验。

如果商品下架、SKU 失效、库存不足或价格变化，应及时提示用户处理。

6. 后端作为最终校验来源

无论前端展示什么，后端订单提交接口都必须作为最终库存校验来源。订单提交时必须重新校验商品状态、SKU 状态、库存数量和购买数量，防止超卖和异常订单。

8.9 总结

综上，用户在商品详情页看到库存为 1，但提交订单时提示 “Insufficient inventory”，可能由多种原因导致，包括商品详情页库存缓存未更新、其他用户抢先下单、SKU 库存与商品总库存不一致、购物车商品状态失效、订单提交 API 重新校验库存，以及前后端库存数据不同步等。

排查该问题时，应按照用户操作路径、接口返回数据、SKU 库存、订单提交 API、库存流水和订单记录逐步分析。核心原则是：前端展示库存只能作为参考，后端实时库存校验才是最终依据。

这类问题不仅需要定位技术原因，也需要优化用户体验。系统应尽量在用户提交订单前提前发现库存变化，并用清晰、友好的提示告诉用户原因，减少用户困惑和交易失败带来的负面体验。

九、核心数据指标设计

本部分主要设计电商系统上线后需要持续监控的核心数据指标。对于一个基础电商 MVP 系统来说，数据指标的作用不仅是观察平台销售表现，更重要的是帮助产品经理判断用户是否能够顺利完成核心交易流程：

浏览商品 → 加入购物车 / 立即购买 → 提交订单 → 完成支付 → 商家发货 → 订单完成

因此，指标设计需要围绕用户转化、交易效率、支付稳定性、商品表现、库存运营和售后问题展开。以下选取 8 个核心指标，作为 MVP 上线后的重点监控对象。

9.1 核心指标表

指标名称	指标含义	计算方式	重要性
商品详情页浏览量	用户查看商品详情页的次数	商品详情页 PV / UV	衡量商品曝光情况和用户浏览兴趣
加购率	浏览商品后加入购物车的用户比例	加购用户数 / 商品详情页访问用户数	衡量商品吸引力和用户购买意向
订单提交转化率	浏览商品后成功提交订单的用户比例	提交订单用户数 / 商品详情页访问用户数	衡量从浏览到下单的转化效果
支付成功率	提交订单后成功完成支付的订单比例	支付成功订单数 / 提交订单数	衡量支付流程是否顺畅、支付系统是否稳定
GMV	已支付订单的总交易金额	已支付订单金额总和	衡量平台整体交易规模
订单完成率	已完成订单占已支付订单的比例	已完成订单数 / 已支付订单数	衡量订单履约和交易闭环完成情况
取消订单率	被取消订单占提交订单的比例	取消订单数 / 提交订单数	帮助发现价格、库存、支付超时或用户犹豫等问题
退款率	退款订单占已支付订单的比例	退款订单数 / 已支付订单数	衡量商品质量、售后体验 and 用户满意度

9.2 商品详情页浏览量

商品详情页浏览量是指用户访问商品详情页的次数，通常可以分为 PV 和 UV 两种统计方式。

- PV 表示页面浏览次数。
- UV 表示访问该页面的独立用户数。

该指标可以帮助产品和运营团队判断商品是否获得了足够曝光。如果商品详情页浏览量较低，可能说明商品列表页排序不合理、搜索结果曝光不足、分类入口不明显，或者商品主图和标题不够吸引用户。

例如，一个商品在列表页中长期没有点击，可能不是商品本身不好，而是商品主图、标题或排序位置存在问题。可以根据该指标进一步分析商品曝光链路。

该指标的重要性在于：它是后续转化漏斗的起点。没有足够的商品详情页访问量，后续的加购、下单和支付都无从谈起。电商转化就像人类找工作，第一步连简历都没人看，后面再优秀也只能在系统里静静发霉。

9.3 加购率

加购率是指用户浏览商品详情页后，将商品加入购物车的比例。

计算方式为：

加购率 = 加购用户数 / 商品详情页访问用户数

加购率可以反映商品对用户的吸引力。如果商品详情页浏览量较高，但加购率较低，说明用户虽然点进来了，但没有产生进一步购买兴趣。

可能原因包括：

- 商品价格不具备竞争力；
- 商品图片或描述不够清晰；
- 库存或规格选择不合理；
- 商品评价、售后政策或配送信息不足；
- 页面购买按钮不明显；
- 商品详情页加载慢或体验差。

对于 MVP 阶段来说，加购率非常重要，因为它能够帮助产品经理判断商品详情页是否有效地推动用户进入购买决策阶段。

如果加购率较低，后续可以优化商品主图、详情文案、价格展示、SKU 选择体验和购买按钮位置。

9.4 订单提交转化率

订单提交转化率是指用户从浏览商品详情页到成功提交订单的比例。

计算方式为：

订单提交转化率 = 提交订单用户数 / 商品详情页访问用户数

该指标用于衡量用户是否能够顺利从商品浏览进入下单流程。相比加购率，订单提交转化率更接近真实购买意向。

如果加购率较高，但订单提交转化率较低，可能说明用户已经有购买兴趣，但在购物车或提交订单环节遇到了阻碍。

可能原因包括：

- 购物车操作复杂；
- 提交订单页面字段过多；
- 收货地址填写流程不顺畅；
- 库存不足；
- 价格或运费在结算页发生变化；
- 登录流程打断用户；
- 页面加载慢或出现报错。

对产品经理来说，该指标可以帮助定位转化漏斗中的问题。如果大量用户停留在加购阶段但不提交订单，就需要重点检查购物车和结算页体验。

9.5 支付成功率

支付成功率是指用户提交订单后，最终成功完成支付的订单比例。

计算方式为：

支付成功率 = 支付成功订单数 / 提交订单数

这是电商系统中非常关键的指标，因为提交订单并不代表平台真正获得交易收入。只有支付成功，订单才进入后续发货和履约阶段。

如果支付成功率较低，可能说明支付流程存在问题。

常见原因包括：

- 支付接口异常；
- 支付渠道不稳定；
- 用户支付超时；
- 支付页面跳转失败；
- 支付方式选择不清晰；
- 订单金额异常；

- 用户在支付前改变购买决策。

MVP 阶段尤其需要重点监控支付成功率，因为支付流程是核心交易闭环中的关键节点。如果用户都提交订单了，却在支付环节大量流失，那就相当于把用户送到收银台再让收银机开始表演罢工，商业系统当场失去尊严。

9.6 GMV

GMV 是 Gross Merchandise Volume 的缩写，指一定时间内平台已支付订单的总交易金额。

计算方式为：

GMV = 已支付订单金额总和

在本项目中，建议只统计已支付订单的金额，而不是所有提交订单的金额。因为未支付订单并没有真正形成交易价值，如果把未支付订单也算入 GMV，会导致数据虚高。

GMV 可以衡量平台整体交易规模，是电商业务最基础的经营指标之一。

通过 GMV，产品和运营团队可以观察：

- 平台整体销售趋势；
- 不同时间段销售变化；
- 不同商品的销售贡献；
- 活动或促销对交易额的影响；
- 用户购买力和客单价变化。

但是需要注意，GMV 不能单独代表业务健康。一个系统可能 GMV 很高，但退款率也很高，或者支付成功率很低。因此，GMV 需要与订单完成率、退款率、支付成功率等指标一起分析。

9.7 订单完成率

订单完成率是指最终进入完成状态的订单，占已支付订单的比例。

计算方式为：

订单完成率 = 已完成订单数 / 已支付订单数

该指标用于衡量订单履约是否顺利。一个订单支付成功后，还需要经过商家发货、物流配送、用户收货或系统自动确认收货，最终才会进入已完成状态。

如果订单完成率较低，可能说明履约流程存在问题。

常见原因包括：

- 商家发货不及时；
- 库存实际不足导致无法发货；
- 物流异常；
- 用户长时间未确认收货；
- 订单被退款或取消；
- 后台订单处理效率低。

对于 MVP 系统来说，订单完成率可以帮助产品经理判断交易闭环是否真正跑通。因为电商系统不只是让用户付钱，还要让用户收到商品。否则这不叫电商，这叫许愿池，还是带支付网关的那种。

9.8 取消订单率

取消订单率是指被取消订单占提交订单总数的比例。

计算方式为：

取消订单率 = 取消订单数 / 提交订单数

取消订单包括用户主动取消和系统自动取消，例如支付超时自动取消。

该指标可以帮助产品经理分析用户为什么没有完成交易。

如果取消订单率较高，可能说明：

- 用户下单后犹豫；
- 支付时间限制不合理；
- 商品价格缺乏吸引力；
- 运费或优惠信息不清晰；
- 库存变化导致订单取消；
- 提交订单后发现地址、规格或金额有问题；
- 支付流程不顺畅导致用户放弃。

建议进一步区分取消原因，例如：

- ☐ 用户主动取消；
- ☐ 支付超时取消；
- ☐ 库存不足取消；
- ☐ 后台取消；
- ☐ 风控取消。

这样可以帮助团队更准确地定位问题，而不是只看到一个总取消率。总取消率只能告诉我们“出事了”，但不能告诉我们“谁干的”，这种数据看了等于白看，还会让会议变长。

9.9 退款率

退款率是指退款订单占已支付订单的比例。

计算方式为：

退款率 = 退款订单数 / 已支付订单数

退款率可以反映商品质量、用户满意度、售后体验和商品信息准确性。

如果退款率较高，可能说明：

- 商品描述与实物不符；
- 商品质量存在问题；
- 用户误购；
- 物流时间过长；
- 售后政策不清晰；
- 用户购买后改变主意；
- 某些商品存在集中投诉。

对于产品经理来说，退款率不仅是售后指标，也可以反向帮助优化商品详情页、SKU 描述、图片展示和购买提示。

例如，如果某个商品退款率明显高于其他商品，就需要进一步分析退款原因。可能是商品本身有质量问题，也可能是详情页描述过于夸张，让用户产生了错误期待。人类很容易被漂亮主图诱惑，然后在收到实物时重新认识现实，这就是退款率存在的意义之一。

9.10 指标漏斗分析

除了单独监控每个指标，还应将这些指标组合成一个完整的交易转化漏斗。

一个基础电商转化漏斗可以设计为：



图9-10 核心转化漏斗图

通过漏斗分析，产品经理可以判断用户主要流失在哪个环节。

- 如果商品详情页浏览量低, 说明曝光不足。
- 如果浏览量高但加购率低, 说明商品吸引力不足或详情页信息不充分。
- 如果加购率高但订单提交转化率低, 说明购物车或提交订单流程存在问题。
- 如果订单提交转化率高但支付成功率低, 说明支付流程可能存在问题。
- 如果支付成功率高但订单完成率低, 说明履约或物流环节可能存在问题。

这种漏斗分析比单独看 GMV 更有价值, 因为它能够帮助团队定位问题发生的具体环节。

9.11 MVP 阶段的数据埋点建议

为了支持上述指标统计, MVP 阶段需要提前设计基础埋点。建议至少记录以下用户行为事件:

- 商品详情页曝光事件: `product_detail_view`
- 加入购物车事件: `add_to_cart`
- 立即购买事件: `buy_now_click`
- 提交订单事件: `submit_order`
- 支付成功事件: `payment_success`
- 支付失败事件: `payment_failed`
- 取消订单事件: `cancel_order`
- 申请退款事件: `request_refund`
- 订单完成事件: `order_completed`

每个事件建议记录基础字段, 例如:

- 用户 ID;
- 商品 ID;
- SKU ID;
- 订单 ID;
- 事件发生时间;
- 页面来源;
- 商品价格;
- 购买数量;
- 订单状态;
- 失败原因或取消原因。

通过这些埋点, 后续可以分析用户行为路径、商品转化效果、支付异常原因和订单履约情况。

9.12 总结

综上，电商系统 MVP 上线后应重点监控商品详情页浏览量、加购率、订单提交转化率、支付成功率、GMV、订单完成率、取消订单率和退款率这 8 个核心指标。

这些指标覆盖了从用户浏览商品到完成交易的完整链路，能够帮助产品经理判断系统是否真正跑通核心交易流程。同时，结合漏斗分析和事件埋点，可以进一步定位用户流失原因和系统异常问题。

对于 MVP 阶段来说，数据指标不需要一开始就非常复杂，但必须能够回答几个关键问题：用户有没有看到商品，用户有没有购买意向，用户能不能顺利下单，支付是否成功，订单是否完成，以及售后问题是否严重。只要这些问题能够被数据回答，产品经理就可以基于真实用户行为进行后续迭代，而不是靠会议室里的想象力指挥系统进化。

十、总结与后续迭代方向

本方案围绕一个基础电商系统的 MVP 版本进行设计，核心目标是优先完成从用户浏览商品到完成订单的基础交易闭环，即：

浏览商品 → 加入购物车 / 立即购买 → 提交订单 → 支付 → 发货 → 完成订单

在产品设计部分，本方案首先明确了系统中的主要用户角色，包括客户 / 买家、运营人员 / 商家人员，以及可选的平台管理员。不同角色在系统中的核心目标不同：客户主要关注浏览、购买、支付和查看订单；运营人员主要关注商品、库存、订单和发货管理；平台管理员则更关注权限、异常订单和系统整体运行情况。

在 MVP 功能规划中，本方案按照 P0、P1、P2 的方式对功能进行了优先级划分。P0 功能主要围绕核心交易流程展开，例如商品列表、商品详情页、购物车、提交订单、支付、订单管理、商品管理和库存管理等。P1 功能主要提升用户体验和运营效率，例如搜索筛选、订单取消、退款处理和数据看板等。P2 功能则可以作为后续版本迭代，例如优惠券、推荐系统、评价系统和更复杂的营销工具。

在流程设计部分，本方案重点梳理了客户下单与支付流程，以及商品创建与发布流程。对于电商系统来说，流程设计不仅要覆盖正常路径，也必须考虑异常情况，例如库存不足、支付失败、支付超时、商品下架但仍存在于购物车等问题。这些异常场景会直接影响用户体验和系统稳定性，因此需要在产品规则、页面提示和后端校验中提前设计清楚。

在 PRD 部分，本方案以具体功能为例，说明了功能背景、用户目标、页面字段、业务规则、异常处理和验收标准。简化版 PRD 的重点不是追求复杂，而是让需求足够清晰，使设计、研发、测试和运营人员都能理解功能应该如何实现、如何判断完成，以及异常情况下系统应该如何响应。

在技术理解部分，本方案通过 API Response、伪代码逻辑和订单状态流转，说明了产品经理需要具备基础的技术理解能力。产品经理不一定需要编写完整代码，但需要理解接口字段含义、价格单位、商品状态、SKU 库存关系、订单状态机和基础业务校验逻辑。只有理解这些内容，才能更准确地与研发沟通，也能避免提出无法落地或边界不清的需求。

在异常分析部分，本方案分析了“商品详情页显示库存为 1，但提交订单时提示库存不足”的可能原因。该问题可能由缓存、并发购买、SKU 库存与商品总库存不一致、购物车商品失效、订单提交 API 重新校验库存，以及前后端数据不同步等原因导致。排查这类问题时，需要按照用户路径、接口数据、SKU 库存、订单日志和库存流水逐步定位。核心原则是：前端展示可以作为用户参考，但后端实时校验必须作为最终判断依据。

在数据指标部分，本方案设计了电商系统上线后需要监控的核心指标，包括商品详情页浏览量、加购率、订单提交转化率、支付成功率、GMV、订单完成率、取消订单率和退款率。这些指标覆盖了从商品曝光到交易完成的完整链路，可以帮助产品经理判断系统是否真正跑通交易闭环，并发现用户流失和系统异常发生在哪个环节。

10.1 MVP 阶段的核心价值

该 MVP 版本的核心价值在于用最小可行功能验证电商系统的基础交易能力。对于客户来说，MVP 版本应保证用户能够顺利完成商品浏览、规格选择、加入购物车、提交订单、支付和查看订单状态等操作。对于运营人员来说，MVP 版本应保证后台可以完成商品创建、库存维护、订单查看和发货管理。对于平台来说，MVP 版本应保证商品、库存、订单、支付和物流状态之间能够形成稳定的数据闭环。因此，MVP 阶段不应过度追求复杂营销功能，而应优先保证核心交易链路

稳定、状态清晰、库存准确、支付可靠。否则优惠券、推荐系统和复杂活动做得再漂亮，用户最后买不了东西，那也只是给系统穿了一件昂贵但没用的外套。

10.2 后续迭代方向

在 MVP 版本上线并验证基础交易流程后，系统可以从以下几个方向进行后续迭代。

1. 提升用户购买体验

后续可以优化商品搜索、分类筛选、商品推荐、商品评价、收藏夹、优惠券和促销活动等功能。这些功能可以帮助用户更快找到商品，并提升购买转化率。

例如，在商品详情页增加用户评价和问答，可以提升用户信任感；在商品列表页增加筛选和排序，可以提升查找效率；在结算页增加优惠券，可以提升用户下单意愿。

2. 优化库存与订单管理能力

MVP 阶段可以先支持基础库存管理，但后续应进一步支持库存预警、库存流水、库存锁定、批量库存调整和库存异常提醒。

订单管理方面，也可以进一步增加订单筛选、批量发货、物流追踪、异常订单标记和售后处理记录，提升运营人员的处理效率。

3. 完善售后与退款流程

MVP 阶段可以先支持基础取消订单和退款状态，但后续应进一步完善退款原因、退款审核、退货物流、售后时限、部分退款和退款失败处理等功能。

售后流程直接影响用户满意度，也会影响平台的长期信任。如果退款流程不清晰，用户很容易从“想买东西”变成“想投诉平台”，人类消费心理就是这么脆弱又合理。

4. 建立更完整的数据分析体系

后续可以基于 MVP 阶段的核心指标，进一步建立数据看板和漏斗分析体系。

例如，可以分析：

- 不同商品的浏览量与转化率；
- 不同渠道的用户转化效果；
- 不同 SKU 的销售表现；
- 支付失败原因分布；

- 订单取消原因分布；
- 退款原因分布；
- 用户复购情况。

这些数据可以帮助产品和运营团队持续优化商品、页面、流程和营销策略。

5. 提升系统稳定性和安全性

随着订单量增长，系统需要进一步关注稳定性和安全性。例如：

- 防止库存超卖；
- 保证支付回调准确；
- 防止重复提交订单；
- 防止恶意刷单；
- 保证用户数据安全；
- 保证订单和支付数据可追踪；
- 增加异常监控和告警机制。

这些能力虽然不一定直接展示在页面上，但会决定系统是否能够长期稳定运行。

10.3 最终总结

总体来看，本方案以电商系统 MVP 为核心，围绕用户角色、功能优先级、核心流程、PRD、API 理解、代码逻辑、订单状态、异常分析和数据指标进行了完整设计。

该方案的重点不是一次性设计一个复杂的大型电商平台，而是优先保证基础交易闭环能够稳定运行。对于 MVP 阶段而言，最重要的是让用户能够买到商品，让运营人员能够管理商品和订单，让系统能够准确处理库存、支付、发货和订单状态。

后续版本可以在 MVP 基础上继续扩展搜索推荐、营销工具、售后体系、数据分析和风控能力。通过这种方式，系统可以先完成从 0 到 1 的核心验证，再逐步从基础可用走向体验优化和运营增长。

因此，本方案的核心思路可以概括为：

1. 先保证交易闭环跑通，再优化用户体验；
2. 先保证库存和订单准确，再扩展营销能力；
3. 先通过 MVP 验证基础价值，再通过数据驱动后续迭代。